# AI Quest: Exam+ questions only (42172 Introduction to AI)

Generated 2026-11-10 from the app's question bank. Each question shows its type, difficulty and source, then the answer and the worked solution.
Types: MCQ (one answer), MULTI (select all that apply), TF (true/false), BLANK (fill one blank), BLANKS (fill several), MATCH (matching).
Part A of each topic = hand-written questions (lectures, workshops, key-points sheets, class exercises, Books 1-3, AIMA). Part B = computer-generated calculation drills (fresh numbers for every skill).

## Contents


- 🧭 Classical Search (Lecture 2): 3 hand-written + 32 calculation drills
- 🌳 Decision Trees (Lecture 4): 0 hand-written + 16 calculation drills
- 🏰 MDPs & Reinforcement Learning (Workshop 6): 0 hand-written + 16 calculation drills
- 🎲 Probability & Bayesian Networks (Lecture 11): 0 hand-written + 16 calculation drills
- 🧱 CNNs & Computer Vision (Lecture 7): 0 hand-written + 16 calculation drills
- 📦 Object Detection (IoU & NMS) (Lecture 7): 0 hand-written + 16 calculation drills
- 🔁 Sequence Models & NLP (Lecture 8): 0 hand-written + 16 calculation drills
- 🎯 Attention & Transformers (Lecture 8): 0 hand-written + 16 calculation drills
- 🧠 Neural Networks (Workshop 5): 0 hand-written + 16 calculation drills


**Total: 163 questions.**

# 🧭 Classical Search (Lecture 2)

## Part A: hand-written questions (3)

### SEARCH A1. [MATCH · Exam+]

Match each algorithm to its entry in the Lecture 2 performance table (Complete? / Cost-optimal?).

1. BFS
2. DFS
3. Greedy best-first
4. A* (admissible h)

A. Complete: No, Optimal: No
B. Complete: Yes*, Optimal: Yes (time O(b^d))
C. Complete: Not always, Optimal: No
D. Complete: No, Optimal: Yes
E. Complete: Yes*, Optimal: Yes

**Answer:** 1 → E, 2 → A, 3 → C, 4 → B

**Working:**

- BFS → Complete: Yes*, Optimal: Yes
- DFS → Complete: No, Optimal: No
- Greedy best-first → Complete: Not always, Optimal: No
- A* (admissible h) → Complete: Yes*, Optimal: Yes (time O(b^d))

**Takeaway:** Only BFS, UCS and A* are cost-optimal in the table.

*Source: Lecture 2, slide 53*

### SEARCH A2. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

```
Node | g | h | f = g + h
Sibiu | 140 | 253 | 393
Timișoara | 118 | 329 | 447
Zerind | 75 | 374 | 449
Râmnicu Vâlcea (via S) | 220 | 193 | 413
Făgăraș (via S) | 239 | 176 | 415
Pitești (via RV) | 317 | 100 | 417
Bucharest (via F) | 450 | 0 | 450
Bucharest (via P) | 418 | 0 | 418
```

In the Romania A* run, when Făgăraș is expanded it adds Bucharest with f = 450 + 0 = 450, so A* returns the 450 km route.

**Answer:** False

**Why:** Bucharest(450) waits in the frontier; Pitești (417) is expanded first and finds Bucharest with f = 418, which is popped before 450.

**Takeaway:** A* returns a goal only when it is selected with the smallest f.

*Source: Lecture 2, slide 48*

### SEARCH A3. [MULTI · Exam+]

Select ALL true statements about A*. (Select ALL that apply.)

A. A* with an inadmissible heuristic is still always optimal
B. f(n) estimates the cost of the cheapest path from start through n to the goal
C. With an admissible heuristic, (tree-search) A* is cost-optimal
D. A* expands the node with the lowest h
E. A* with h(n) = 0 behaves like uniform-cost search

**Answer:** B. f(n) estimates the cost of the cheapest path from start through n to the goal; C. With an admissible heuristic, (tree-search) A* is cost-optimal; E. A* with h(n) = 0 behaves like uniform-cost search

**Option notes:**

- A: Incorrect: Overestimates can make A* skip the optimal path.
- B: Correct: Slide 47.
- C: Correct: Slide 50/52.
- D: Incorrect: That's greedy; A* uses g + h.
- E: Correct: f = g + 0 = g.

**Takeaway:** h = 0 turns A* into UCS.

*Source: Lecture 2, slide 47-52*

## Part B: calculation drills (32)

### SEARCH B1. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=8, A h=5, B h=4, C h=2, D h=2, E h=4, F h=0, H h=0, G h=0

True or False: the heuristic h on the graph is consistent, i.e. $h(n) \le c(n, n') + h(n')$ for every edge (both directions).

**Answer:** True

**Why:** Every edge satisfies the triangle inequality.

**Working:**

- Check $h(n) \le c(n,n') + h(n')$ on every edge, in both directions.
- S–A (c=4): $8 \le 4+5=9$ ✓; $5 \le 4+8=12$ ✓
- S–B (c=4): $8 \le 4+4=8$ ✓; $4 \le 4+8=12$ ✓
- A–C (c=7): $5 \le 7+2=9$ ✓; $2 \le 7+5=12$ ✓
- A–E (c=9): $5 \le 9+4=13$ ✓; $4 \le 9+5=14$ ✓
- B–D (c=7): $4 \le 7+5=12$ ✓; $5 \le 7+4=11$ ✓
- B–E (c=8): $4 \le 8+4=12$ ✓; $4 \le 8+4=12$ ✓

- C–F (c=8): 2 ≤ 8+0=8 ✓; 0 ≤ 8+2=10 ✓
- C–H (c=2): 2 ≤ 2+0=2 ✓; 0 ≤ 2+2=4 ✓
- D–F (c=7): 5 ≤ 7+0=7 ✓; 0 ≤ 7+5=12 ✓
- E–H (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- F–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- H–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- B–H (c=4): 4 ≤ 4+0=4 ✓; 0 ≤ 4+4=8 ✓
- All pass → consistent (and therefore also admissible).

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B2. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=14, A h=10, B h=14, C h=17, D h=17, E h=14, F h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge E→A: h(E) = 14 > 1 + h(A) = 11.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=1): 14 ≤ 1+14=15 ✓; 14 ≤ 1+14=15 ✓
- S–A (c=4): 14 ≤ 4+10=14 ✓; 10 ≤ 4+14=18 ✓
- A–C (c=7): 10 ≤ 7+17=24 ✓; 17 ≤ 7+10=17 ✓
- A–E (c=1): 10 ≤ 1+14=15 ✓; 14 ≤ 1+10=11 ✗
- B–C (c=6): 14 ≤ 6+17=23 ✓; 17 ≤ 6+14=20 ✓
- B–D (c=9): 14 ≤ 9+11=20 ✓; 11 ≤ 9+14=23 ✓
- C–F (c=9): 17 ≤ 9+6=15 ✗; 6 ≤ 9+17=26 ✓
- D–F (c=7): 11 ≤ 7+6=13 ✓; 6 ≤ 7+11=18 ✓
- E–F (c=6): 14 ≤ 6+6=12 ✗; 6 ≤ 6+14=20 ✓
- F–G (c=9): 6 ≤ 9+0=9 ✓; 0 ≤ 9+6=15 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B3. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=9, A h=9, B h=7, C h=7, D h=7, E h=4, F h=2, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge A→D: h(A) = 9 > 1 + h(D) = 8.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=6): 9 ≤ 6+9=15 ✓; 9 ≤ 6+9=15 ✓
- S–B (c=6): 9 ≤ 6+7=13 ✓; 7 ≤ 6+9=15 ✓
- A–D (c=1): 9 ≤ 1+7=8 ✗; 7 ≤ 1+9=10 ✓
- A–E (c=9): 9 ≤ 9+4=13 ✓; 4 ≤ 9+9=18 ✓
- B–D (c=7): 7 ≤ 7+7=14 ✓; 7 ≤ 7+7=14 ✓
- B–E (c=2): 7 ≤ 2+4=6 ✗; 4 ≤ 2+7=9 ✓
- A–C (c=1): 9 ≤ 1+7=8 ✗; 7 ≤ 1+9=10 ✓
- C–F (c=7): 7 ≤ 7+2=9 ✓; 2 ≤ 7+7=14 ✓
- D–F (c=6): 7 ≤ 6+2=8 ✓; 2 ≤ 6+7=13 ✓
- E–F (c=5): 4 ≤ 5+2=7 ✓; 2 ≤ 5+4=9 ✓
- F–G (c=3): 2 ≤ 3+0=3 ✓; 0 ≤ 3+2=5 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B4. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=2, D h=7, E h=10, F h=7, H h=1, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge E→A: h(E) = 10 > 4 + h(A) = 8.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=2): 4 ≤ 2+2=4 ✓; 2 ≤ 2+4=6 ✓
- S–A (c=1): 4 ≤ 1+4=5 ✓; 4 ≤ 1+4=5 ✓
- A–E (c=4): 4 ≤ 4+10=14 ✓; 10 ≤ 4+4=8 ✗
- A–D (c=6): 4 ≤ 6+7=13 ✓; 7 ≤ 6+4=10 ✓
- B–D (c=8): 2 ≤ 8+7=15 ✓; 7 ≤ 8+2=10 ✓
- B–C (c=1): 2 ≤ 1+2=3 ✓; 2 ≤ 1+2=3 ✓
- C–H (c=1): 2 ≤ 1+1=2 ✓; 1 ≤ 1+2=3 ✓
- C–F (c=2): 2 ≤ 2+7=9 ✓; 7 ≤ 2+2=4 ✗
- D–F (c=3): 7 ≤ 3+7=10 ✓; 7 ≤ 3+7=10 ✓
- E–H (c=9): 10 ≤ 9+1=10 ✓; 1 ≤ 9+10=19 ✓
- F–G (c=7): 7 ≤ 7+0=7 ✓; 0 ≤ 7+7=14 ✓
- H–G (c=2): 1 ≤ 2+0=2 ✓; 0 ≤ 2+1=3 ✓
- B–F (c=4): 2 ≤ 4+7=11 ✓; 7 ≤ 4+2=6 ✗
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B5. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=4, A h=8, B h=8, C h=6, D h=3, E h=3, F h=2, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge A→S: h(A) = 8 > 2 + h(S) = 6.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 4 ≤ 6+8=14 ✓; 8 ≤ 6+4=10 ✓
- S–A (c=2): 4 ≤ 2+8=10 ✓; 8 ≤ 2+4=6 ✗
- A–D (c=1): 8 ≤ 1+3=4 ✗; 3 ≤ 1+8=9 ✓
- A–E (c=2): 8 ≤ 2+7=9 ✓; 7 ≤ 2+8=10 ✓
- B–C (c=4): 8 ≤ 4+6=10 ✓; 6 ≤ 4+8=12 ✓
- B–D (c=6): 8 ≤ 6+3=9 ✓; 3 ≤ 6+8=14 ✓
- C–F (c=8): 6 ≤ 8+2=10 ✓; 2 ≤ 8+6=14 ✓
- D–F (c=2): 3 ≤ 2+2=4 ✓; 2 ≤ 2+3=5 ✓
- E–F (c=9): 7 ≤ 9+2=11 ✓; 2 ≤ 9+7=16 ✓
- F–G (c=3): 2 ≤ 3+0=3 ✓; 0 ≤ 3+2=5 ✓
- A–F (c=4): 8 ≤ 4+2=6 ✗; 2 ≤ 4+8=12 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B6. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=7, B h=5, C h=2, D h=2, E h=5, F h=11, H h=1, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge D→B: h(D) = 8 > 1 + h(B) = 6.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=3): 8 ≤ 3+5=8 ✓; 5 ≤ 3+8=11 ✓
- S–A (c=2): 8 ≤ 2+7=9 ✓; 7 ≤ 2+8=10 ✓
- A–E (c=6): 7 ≤ 6+5=11 ✓; 5 ≤ 6+7=13 ✓
- B–D (c=1): 5 ≤ 1+8=9 ✓; 8 ≤ 1+5=6 ✗
- B–E (c=8): 5 ≤ 8+5=13 ✓; 5 ≤ 8+5=13 ✓
- B–C (c=6): 5 ≤ 6+2=8 ✓; 2 ≤ 6+5=11 ✓
- C–F (c=5): 2 ≤ 5+11=16 ✓; 11 ≤ 5+2=7 ✗
- C–H (c=1): 2 ≤ 1+1=2 ✓; 1 ≤ 1+2=3 ✓
- D–H (c=9): 8 ≤ 9+1=10 ✓; 1 ≤ 9+8=17 ✓

- E–F (c=7): 5 ≤ 7+11=18 ✓; 11 ≤ 7+5=12 ✓
- E–H (c=4): 5 ≤ 4+1=5 ✓; 1 ≤ 4+5=9 ✓
- F–G (c=7): 11 ≤ 7+0=7 ✗; 0 ≤ 7+11=18 ✓
- H–G (c=2): 1 ≤ 2+0=2 ✓; 0 ≤ 2+1=3 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B7. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=6, C h=8, D h=9, E h=5, F h=3, H h=3, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge D→H: h(D) = 9 > 5 + h(H) = 8.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=1): 6 ≤ 1+7=8 ✓; 7 ≤ 1+6=7 ✓
- S–B (c=1): 6 ≤ 1+6=7 ✓; 6 ≤ 1+6=7 ✓
- A–D (c=8): 7 ≤ 8+9=17 ✓; 9 ≤ 8+7=15 ✓
- B–E (c=1): 6 ≤ 1+5=6 ✓; 5 ≤ 1+6=7 ✓
- B–C (c=8): 6 ≤ 8+8=16 ✓; 8 ≤ 8+6=14 ✓
- C–H (c=9): 8 ≤ 9+3=12 ✓; 3 ≤ 9+8=17 ✓
- C–F (c=6): 8 ≤ 6+3=9 ✓; 3 ≤ 6+8=14 ✓
- D–F (c=9): 9 ≤ 9+3=12 ✓; 3 ≤ 9+9=18 ✓
- D–H (c=5): 9 ≤ 5+3=8 ✗; 3 ≤ 5+9=14 ✓
- E–F (c=1): 5 ≤ 1+3=4 ✗; 3 ≤ 1+5=6 ✓
- F–G (c=5): 3 ≤ 5+0=5 ✓; 0 ≤ 5+3=8 ✓
- H–G (c=5): 3 ≤ 5+0=5 ✓; 0 ≤ 5+3=8 ✓
- B–F (c=3): 6 ≤ 3+3=6 ✓; 3 ≤ 3+6=9 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B8. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=9, A h=7, B h=6, C h=4, D h=3, E h=3, F h=3, H h=3, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge B→E: h(B) = 6 > 1 + h(E) = 4.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=8): 9 ≤ 8+7=15 ✓; 7 ≤ 8+9=17 ✓
- S–B (c=9): 9 ≤ 9+6=15 ✓; 6 ≤ 9+9=18 ✓
- A–C (c=4): 7 ≤ 4+4=8 ✓; 4 ≤ 4+7=11 ✓
- B–D (c=4): 6 ≤ 4+3=7 ✓; 3 ≤ 4+6=10 ✓
- B–E (c=1): 6 ≤ 1+3=4 ✗; 3 ≤ 1+6=7 ✓
- C–F (c=3): 4 ≤ 3+3=6 ✓; 3 ≤ 3+4=7 ✓
- C–H (c=8): 4 ≤ 8+3=11 ✓; 3 ≤ 8+4=12 ✓
- D–H (c=6): 3 ≤ 6+3=9 ✓; 3 ≤ 6+3=9 ✓
- D–F (c=3): 3 ≤ 3+3=6 ✓; 3 ≤ 3+3=6 ✓
- E–H (c=2): 3 ≤ 2+3=5 ✓; 3 ≤ 2+3=5 ✓
- E–F (c=2): 3 ≤ 2+3=5 ✓; 3 ≤ 2+3=5 ✓
- F–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- H–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B9. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6) Heuristic values: S h=11, A h=12, B h=9, C h=5, D h=4, E h=9, F h=2, H h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge S→B: h(S) = 11 > 1 + h(B) = 10.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=1): 11 ≤ 1+9=10 ✗; 9 ≤ 1+11=12 ✓
- S–A (c=2): 11 ≤ 2+12=14 ✓; 12 ≤ 2+11=13 ✓
- A–C (c=9): 12 ≤ 9+5=14 ✓; 5 ≤ 9+12=21 ✓
- A–D (c=9): 12 ≤ 9+4=13 ✓; 4 ≤ 9+12=21 ✓
- B–E (c=1): 9 ≤ 1+9=10 ✓; 9 ≤ 1+9=10 ✓
- B–C (c=5): 9 ≤ 5+5=10 ✓; 5 ≤ 5+9=14 ✓
- C–F (c=2): 5 ≤ 2+2=4 ✗; 2 ≤ 2+5=7 ✓
- C–H (c=5): 5 ≤ 5+6=11 ✓; 6 ≤ 5+5=10 ✓
- D–F (c=2): 4 ≤ 2+2=4 ✓; 2 ≤ 2+4=6 ✓
- D–H (c=9): 4 ≤ 9+6=15 ✓; 6 ≤ 9+4=13 ✓
- E–F (c=6): 9 ≤ 6+2=8 ✗; 2 ≤ 6+9=15 ✓
- E–H (c=3): 9 ≤ 3+6=9 ✓; 6 ≤ 3+9=12 ✓
- F–G (c=4): 2 ≤ 4+0=4 ✓; 0 ≤ 4+2=6 ✓
- H–G (c=8): 6 ≤ 8+0=8 ✓; 0 ≤ 8+6=14 ✓
- B–H (c=6): 9 ≤ 6+6=12 ✓; 6 ≤ 6+9=15 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B10. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=13, C h=11, D h=12, E h=12, F h=7, H h=5, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge A→E: h(A) = 17 > 4 + h(E) = 12.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=7): 19 ≤ 7+13=20 ✓; 13 ≤ 7+19=26 ✓
- S–A (c=3): 19 ≤ 3+17=20 ✓; 17 ≤ 3+19=22 ✓
- A–E (c=4): 17 ≤ 4+8=12 ✗; 8 ≤ 4+17=21 ✓
- B–C (c=4): 13 ≤ 4+11=15 ✓; 11 ≤ 4+13=17 ✓
- B–D (c=7): 13 ≤ 7+12=19 ✓; 12 ≤ 7+13=20 ✓
- C–H (c=5): 11 ≤ 5+5=10 ✗; 5 ≤ 5+11=16 ✓
- D–H (c=6): 12 ≤ 6+5=11 ✗; 5 ≤ 6+12=18 ✓
- E–H (c=6): 8 ≤ 6+5=11 ✓; 5 ≤ 6+8=14 ✓
- E–F (c=5): 8 ≤ 5+7=12 ✓; 7 ≤ 5+8=13 ✓
- F–G (c=9): 7 ≤ 9+0=9 ✓; 0 ≤ 9+7=16 ✓
- H–G (c=8): 5 ≤ 8+0=8 ✓; 0 ≤ 8+5=13 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B11. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=5, A h=7, B h=7, C h=9, D h=5, E h=5, F h=3, H h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge A→S: h(A) = 7 > 1 + h(S) = 6.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=1): 5 ≤ 1+7=8 ✓; 7 ≤ 1+5=6 ✗
- S–B (c=9): 5 ≤ 9+7=16 ✓; 7 ≤ 9+5=14 ✓
- A–E (c=4): 7 ≤ 4+6=10 ✓; 6 ≤ 4+7=11 ✓
- A–C (c=8): 7 ≤ 8+9=17 ✓; 9 ≤ 8+7=15 ✓
- B–C (c=1): 7 ≤ 1+9=10 ✓; 9 ≤ 1+7=8 ✗

- B–E (c=9): 7 ≤ 9+6=15 ✓; 6 ≤ 9+7=16 ✓
- A–D (c=2): 7 ≤ 2+5=7 ✓; 5 ≤ 2+7=9 ✓
- C–H (c=5): 9 ≤ 5+4=9 ✓; 4 ≤ 5+9=14 ✓
- D–F (c=5): 5 ≤ 5+3=8 ✓; 3 ≤ 5+5=10 ✓
- D–H (c=3): 5 ≤ 3+4=7 ✓; 4 ≤ 3+5=8 ✓
- E–H (c=6): 6 ≤ 6+4=10 ✓; 4 ≤ 6+6=12 ✓
- E–F (c=4): 6 ≤ 4+3=7 ✓; 3 ≤ 4+6=10 ✓
- F–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- H–G (c=5): 4 ≤ 5+0=5 ✓; 0 ≤ 5+4=9 ✓
- A–F (c=5): 7 ≤ 5+3=8 ✓; 3 ≤ 5+7=12 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B12. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=16, A h=7, B h=9, C h=7, D h=5, E h=5, F h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge S→B: h(S) = 16 > 6 + h(B) = 15.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 16 ≤ 6+9=15 ✗; 9 ≤ 6+16=22 ✓
- S–A (c=7): 16 ≤ 7+7=14 ✗; 7 ≤ 7+16=23 ✓
- A–E (c=2): 7 ≤ 2+9=11 ✓; 9 ≤ 2+7=9 ✓
- A–D (c=9): 7 ≤ 9+5=14 ✓; 5 ≤ 9+7=16 ✓
- B–D (c=5): 9 ≤ 5+5=10 ✓; 5 ≤ 5+9=14 ✓
- B–C (c=2): 9 ≤ 2+7=9 ✓; 7 ≤ 2+9=11 ✓
- C–F (c=3): 7 ≤ 3+4=7 ✓; 4 ≤ 3+7=10 ✓
- D–F (c=3): 5 ≤ 3+4=7 ✓; 4 ≤ 3+5=8 ✓
- E–F (c=5): 9 ≤ 5+4=9 ✓; 4 ≤ 5+9=14 ✓
- F–G (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B13. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=12, A h=11, B h=20, C h=10, D h=12, E h=10, F h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge B→S: h(B) = 20 > 5 + h(S) = 17.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=6): 12 ≤ 6+11=17 ✓; 11 ≤ 6+12=18 ✓
- S–B (c=5): 12 ≤ 5+20=25 ✓; 20 ≤ 5+12=17 ✗
- A–D (c=1): 11 ≤ 1+12=13 ✓; 12 ≤ 1+11=12 ✓
- A–C (c=7): 11 ≤ 7+10=17 ✓; 10 ≤ 7+11=18 ✓
- B–E (c=7): 20 ≤ 7+10=17 ✗; 10 ≤ 7+20=27 ✓
- C–F (c=6): 10 ≤ 6+6=12 ✓; 6 ≤ 6+10=16 ✓
- D–F (c=6): 12 ≤ 6+6=12 ✓; 6 ≤ 6+12=18 ✓
- E–F (c=9): 10 ≤ 9+6=15 ✓; 6 ≤ 9+10=19 ✓
- F–G (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B14. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=9, A h=12, B h=6, C h=8, D h=8, E h=13, F h=4, H h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge C→F: h(C) = 8 > 3 + h(F) = 7.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=3): 9 ≤ 3+6=9 ✓; 6 ≤ 3+9=12 ✓
- S–A (c=6): 9 ≤ 6+12=18 ✓; 12 ≤ 6+9=15 ✓
- A–E (c=4): 12 ≤ 4+13=17 ✓; 13 ≤ 4+12=16 ✓
- B–C (c=2): 6 ≤ 2+8=10 ✓; 8 ≤ 2+6=8 ✓
- B–D (c=3): 6 ≤ 3+7=10 ✓; 7 ≤ 3+6=9 ✓
- C–F (c=3): 8 ≤ 3+4=7 ✗; 4 ≤ 3+8=11 ✓
- D–F (c=6): 7 ≤ 6+4=10 ✓; 4 ≤ 6+7=13 ✓
- D–H (c=7): 7 ≤ 7+4=11 ✓; 4 ≤ 7+7=14 ✓
- E–F (c=9): 13 ≤ 9+4=13 ✓; 4 ≤ 9+13=22 ✓
- F–G (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- H–G (c=5): 4 ≤ 5+0=5 ✓; 0 ≤ 5+4=9 ✓
- B–H (c=7): 6 ≤ 7+4=11 ✓; 4 ≤ 7+6=13 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B15. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)
Heuristic values: S h=11, A h=14, B h=5, C h=7, D h=9, E h=9, F h=5, H h=7, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge C→B: h(C) = 7 > 1 + h(B) = 6.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 11 ≤ 6+5=11 ✓; 5 ≤ 6+11=17 ✓
- S–A (c=3): 11 ≤ 3+14=17 ✓; 14 ≤ 3+11=14 ✓
- A–D (c=7): 14 ≤ 7+9=16 ✓; 9 ≤ 7+14=21 ✓
- A–E (c=5): 14 ≤ 5+12=17 ✓; 12 ≤ 5+14=19 ✓
- B–D (c=9): 5 ≤ 9+9=18 ✓; 9 ≤ 9+5=14 ✓
- B–C (c=1): 5 ≤ 1+7=8 ✓; 7 ≤ 1+5=6 ✗
- C–F (c=1): 7 ≤ 1+5=6 ✗; 5 ≤ 1+7=8 ✓
- C–H (c=1): 7 ≤ 1+7=8 ✓; 7 ≤ 1+7=8 ✓
- D–H (c=8): 9 ≤ 8+7=15 ✓; 7 ≤ 8+9=17 ✓
- E–F (c=8): 12 ≤ 8+5=13 ✓; 5 ≤ 8+12=20 ✓
- E–H (c=7): 12 ≤ 7+7=14 ✓; 7 ≤ 7+12=19 ✓
- F–G (c=8): 5 ≤ 8+0=8 ✓; 0 ≤ 8+5=13 ✓
- H–G (c=8): 7 ≤ 8+0=8 ✓; 0 ≤ 8+7=15 ✓
- B–F (c=1): 5 ≤ 1+5=6 ✓; 5 ≤ 1+5=6 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B16. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=13, A h=9, B h=9, C h=6, D h=10, E h=3, F h=6, H h=0, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

**Answer:** False

**Why:** Edge S→A: h(S) = 13 > 3 + h(A) = 12.

**Working:**

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=8): 13 ≤ 8+9=17 ✓; 9 ≤ 8+13=21 ✓
- S–A (c=3): 13 ≤ 3+9=12 ✗; 9 ≤ 3+13=16 ✓
- A–E (c=9): 9 ≤ 9+3=12 ✓; 3 ≤ 9+9=18 ✓
- A–C (c=7): 9 ≤ 7+6=13 ✓; 6 ≤ 7+9=16 ✓

- B–E (c=7): 9 ≤ 7+3=10 ✓; 3 ≤ 7+9=16 ✓
- B–C (c=6): 9 ≤ 6+6=12 ✓; 6 ≤ 6+9=15 ✓
- B–D (c=1): 9 ≤ 1+10=11 ✓; 10 ≤ 1+9=10 ✓
- C–F (c=4): 6 ≤ 4+6=10 ✓; 6 ≤ 4+6=10 ✓
- C–H (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- D–F (c=4): 10 ≤ 4+6=10 ✓; 6 ≤ 4+10=14 ✓
- E–F (c=3): 3 ≤ 3+6=9 ✓; 6 ≤ 3+3=6 ✓
- E–H (c=3): 3 ≤ 3+0=3 ✓; 0 ≤ 3+3=6 ✓
- F–G (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- H–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- At least one edge fails → NOT consistent.

**Takeaway:** Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B17. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (7); A — E (9); A — C (9); B — D (8); B — C (7); C — F (8); C — H (5); D — H (5); E — H (4); E — F (7); F — G (3); H — G (5); A — H (5)
Heuristic values: S h=10, A h=9, B h=13, C h=7, D h=7, E h=5, F h=1, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 12; Greedy cost = 12; A* cost = 12

**Working:**

- UCS order S,A,B,H,E,C,D,G → path S-A-H-G, cost 12.
- Greedy order S,A,H,G → path S-A-H-G, cost 12.
- A* order S,A,H,G → path S-A-H-G, cost 12.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 12
  Greedy | 4 | 12
  A* | 4 | 12

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B18. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (3); A — D (3); B — C (6); B — E (9); C — H (6); D — H (4); E — H (3); C — F (5); F — G (1); H — G (7)
Heuristic values: S h=11, A h=9, B h=11, C h=3, D h=3, E h=5, F h=0, H h=5, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 15; Greedy cost = 23; A* cost = 15

**Working:**

- UCS order S,B,A,C,D,E,F,G → path S-B-C-F-G, cost 15.
- Greedy order S,A,D,H,G → path S-A-D-H-G, cost 23.
- A* order S,B,C,F,G → path S-B-C-F-G, cost 15.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 23
  A* | 5 | 15

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B19. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (2); A — C (2); B — E (5); B — D (9); C — F (1); D — F (5); E — F (4); F — G (7)
Heuristic values: S h=10, A h=6, B h=14, C h=5, D h=8, E h=10, F h=4, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 15; Greedy cost = 15; A* cost = 15

**Working:**

- UCS order S,B,A,C,E,F,D,G → path S-A-C-F-G, cost 15.
- Greedy order S,A,C,F,G → path S-A-C-F-G, cost 15.
- A* order S,A,C,F,G → path S-A-C-F-G, cost 15.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 15
  A* | 5 | 15

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B20. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — C (1); B — E (2); B — C (1); B — D (6); C — H (9); C — F (2); D — H (3); E — F (8); F — G (5); H — G (7)
Heuristic values: S h=12, A h=6, B h=4, C h=3, D h=8, E h=5, F h=4, H h=6, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry

is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 15; Greedy cost = 16; A* cost = 16

**Working:**

- UCS order S,A,B,C,E,F,D,G → path S-A-C-F-G, cost 15.
- Greedy order S,B,C,F,G → path S-B-C-F-G, cost 16.
- A* order S,B,C,A,E,F,G → path S-B-C-F-G, cost 16.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 16
  A* | 7 | 16

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B21. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — D (8); A — E (4); B — E (6); B — C (2); C — F (4); C — H (5); D — H (7); E — F (2); E — H (6); F — G (5); H — G (4); B — H (2)
Heuristic values: S h=11, A h=11, B h=5, C h=5, D h=4, E h=4, F h=3, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 14; Greedy cost = 14; A* cost = 14

**Working:**

- UCS order S,A,B,C,H,E,D,F,G → path S-B-H-G, cost 14.
- Greedy order S,B,H,G → path S-B-H-G, cost 14.
- A* order S,B,H,G → path S-B-H-G, cost 14.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 14
  Greedy | 4 | 14
  A* | 4 | 14

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B22. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (4); S — A (6); A — E (8); A — D (4); B — C (5); B — D (7); C — F (7); D — F (3); E — F (5); F — G (5)
Heuristic values: S h=10, A h=9, B h=14, C h=11, D h=5, E h=5, F h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 19; Greedy cost = 22; A* cost = 19

**Working:**

- UCS order S,B,A,C,D,E,F,G → path S-B-D-F-G, cost 19.
- Greedy order S,A,D,F,G → path S-A-D-F-G, cost 22.
- A* order S,A,E,B,D,F,C,G → path S-B-D-F-G, cost 19.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 19
  Greedy | 5 | 22
  A* | 8 | 19

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B23. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (5); A — C (7); B — C (1); B — E (3); A — D (2); C — H (1); C — F (4); D — F (4); E — F (6); F — G (7); H — G (4); B — H (5)
Heuristic values: S h=10, A h=8, B h=6, C h=4, D h=7, E h=7, F h=4, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 12; Greedy cost = 14; A* cost = 12

**Working:**

- UCS order S,B,A,C,D,E,H,F,G → path S-B-C-H-G, cost 12.
- Greedy order S,B,H,G → path S-B-H-G, cost 14.
- A* order S,B,C,H,G → path S-B-C-H-G, cost 12.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 12
  Greedy | 4 | 14
  A* | 5 | 12

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B24. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (1); A — E (8); A — D (8); B — C (1); B — E (9); C — F (3); C — H (1); D — F (1); D — H (4); E — H (4); F — G (9); H — G (1)

Heuristic values: S h=6, A h=3, B h=7, C h=1, D h=3, E h=3, F h=4, H h=0, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 7; Greedy cost = 14; A* cost = 7

**Working:**

- UCS order S,A,E,B,H,C,G → path S-A-E-H-G, cost 7.
- Greedy order S,A,D,H,G → path S-A-D-H-G, cost 14.
- A* order S,A,E,H,G → path S-A-E-H-G, cost 7.
- Algorithm | Nodes expanded | Cost
  UCS | 7 | 7
  Greedy | 5 | 14
  A* | 5 | 7

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B25. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (8); A — E (4); B — C (9); B — E (6); B — D (6); C — F (1); D — F (2); E — F (6); F — G (6); A — F
Heuristic values: S h=10, A h=8, B h=9, C h=6, D h=7, E h=6, F h=5, G h=0
Other labels: 9

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 17; Greedy cost = 17; A* cost = 17

**Working:**

- UCS order S,A,B,E,F,C,D,G → path S-A-F-G, cost 17.
- Greedy order S,A,F,G → path S-A-F-G, cost 17.
- A* order S,A,F,B,E,G → path S-A-F-G, cost 17.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 17
  Greedy | 4 | 17
  A* | 6 | 17

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B26. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (9); A — C (3); A — D (8); B — C (5); B — E (6); C — H (2); D — H (7); D — F (7); E — H (6); E — F (8); F — G (9); H — G (3); A — H
Heuristic values: S h=10, A h=6, B h=7, C h=4, D h=4, E h=5, F h=8, H h=2, G h=0
Other labels: 7

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 17; Greedy cost = 17; A* cost = 17

**Working:**

- UCS order S,A,B,C,H,E,D,G → path S-A-H-G, cost 17.
- Greedy order S,A,H,G → path S-A-H-G, cost 17.
- A* order S,A,B,C,H,G → path S-A-H-G, cost 17.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 17
  Greedy | 4 | 17
  A* | 6 | 17

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B27. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (6); A — D (2); B — E (4); B — D (9); B — C (5); C — F (1); C — H (6); D — H (8); E — F (8); F — G (1); H — G (8); A — F (4)
Heuristic values: S h=7, A h=3, B h=6, C h=1, D h=5, E h=6, F h=0, H h=7, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 11; Greedy cost = 11; A* cost = 11

**Working:**

- UCS order S,B,A,D,E,C,F,G → path S-A-F-G, cost 11.
- Greedy order S,A,F,G → path S-A-F-G, cost 11.
- A* order S,A,F,B,C,G → path S-A-F-G, cost 11.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 11
  Greedy | 4 | 11
  A* | 6 | 11

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B28. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (5); A — D (8); B — C (9); B — E (2); C — F (6); D — F (4); E — F (8); F — G (3); A — F (6)
Heuristic values: S h=11, A h=8, B h=8, C h=6, D h=5, E h=5, F h=1, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 18; Greedy cost = 18; A* cost = 18

**Working:**

- UCS order S,B,E,A,C,F,D,G → path S-B-E-F-G, cost 18.
- Greedy order S,A,F,G → path S-A-F-G, cost 18.
- A* order S,B,A,F,E,G → path S-A-F-G, cost 18.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 18
  Greedy | 4 | 18
  A* | 6 | 18

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B29. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — E (7); A — D (4); B — C (6); B — E (2); C — H (6); D — F (8); D — H (7); E — F (7); E — H (1); F — G (3); H — G (3); A — H (1)
Heuristic values: S h=6, A h=2, B h=4, C h=6, D h=6, E h=3, F h=2, H h=2, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 10; Greedy cost = 10; A* cost = 10

**Working:**

- UCS order S,A,B,H,E,D,G → path S-A-H-G, cost 10.
- Greedy order S,A,H,G → path S-A-H-G, cost 10.
- A* order S,A,H,B,G → path S-A-H-G, cost 10.
- Algorithm | Nodes expanded | Cost
  UCS | 7 | 10
  Greedy | 4 | 10
  A* | 5 | 10

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B30. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (3); A — D (3); A — C (8); B — C (8); B — E (4); C — F (3); D — F (2); E — F (9); F — G (6)
Heuristic values: S h=15, A h=10, B h=10, C h=8, D h=7, E h=13, F h=4, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 19; Greedy cost = 19; A* cost = 19

**Working:**

- UCS order S,B,E,A,C,D,F,G → path S-A-D-F-G, cost 19.
- Greedy order S,A,D,F,G → path S-A-D-F-G, cost 19.
- A* order S,B,A,D,F,C,G → path S-A-D-F-G, cost 19.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 19
  Greedy | 5 | 19
  A* | 7 | 19

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

## SEARCH B31. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (1); A — E (1); A — D (6); B — D (1); B — C (5); C — F (8); C — H (1); D — H (9); E — H (3); E — F (2); F — G (4); H — G (4); B — F (8)
Heuristic values: S h=6, A h=4, B h=6, C h=4, D h=6, E h=6, F h=3, H h=2, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 11; Greedy cost = 16; A* cost = 11

**Working:**

- UCS order S,B,D,C,H,A,E,F,G → path S-B-C-H-G, cost 11.
- Greedy order S,A,E,H,G → path S-A-E-H-G, cost 16.
- A* order S,B,D,C,H,G → path S-B-C-H-G, cost 11.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 11

Greedy | 5 | 16
A* | 6 | 11

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

### SEARCH B32. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (3); A — E (3); A — D (3); B — E (3); A — C; C — F (5); D — F (1); E — F (2); F — G (9)
Heuristic values: S h=9, A h=10, B h=10, C h=11, D h=9, E h=9, F h=8, G h=0
Other labels: 9

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

**Answer:** UCS cost = 16; Greedy cost = 23; A* cost = 16

**Working:**

- UCS order S,A,C,D,F,B,E,G → path S-A-D-F-G, cost 16.
- Greedy order S,A,E,F,G → path S-A-E-F-G, cost 23.
- A* order S,A,D,F,E,G → path S-A-D-F-G, cost 16.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 16
  Greedy | 5 | 23
  A* | 6 | 16

**Takeaway:** UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

*Source: Lecture 2 (Classic search) + UCS/GBFS/A* Romania examples*

---

# 🌳 Decision Trees (Lecture 4)

## Part B: calculation drills (16)

### TREES B1. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [6+, 9−] · Low · [0+, 4−] · Medium · [4+, 1−] · High · [2+, 4−]

```
[Diagram as text] Labels: Student? · [6+, 9−] · a · [2+, 7−] · b · [4+, 2−]
```

Two candidate splits of [6+, 9−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.363; IG(Student) = 0.145; Chosen attribute = Colour

**Working:**

- H(parent [6+, 9−]) = −(6/15)·$\log_2$(6/15) = 0.5288 + −(9/15)·$\log_2$(9/15) = 0.4422 = 0.9710
- H(Low [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- H(Medium [4+, 1−]) = −(4/5)·$\log_2$(4/5) = 0.2575 + −(1/5)·$\log_2$(1/5) = 0.4644 = 0.7219
- H(High [2+, 4−]) = −(2/6)·$\log_2$(2/6) = 0.5283 + −(4/6)·$\log_2$(4/6) = 0.3900 = 0.9183
- Weighted child entropy = (4/15)·0.0000 + (5/15)·0.7219 + (6/15)·0.9183 = 0.6080
- IG(Colour) = 0.9710 − 0.6080 = 0.3630
- H(Student=a [2+, 7−]) = −(2/9)·$\log_2$(2/9) = 0.4822 + −(7/9)·$\log_2$(7/9) = 0.2820 = 0.7642
- H(Student=b [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- Weighted child entropy (Student) = 0.8258 → IG(Student) = 0.9710 − 0.8258 = 0.1451
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.2844, Student = 0.3852 → Gini prefers Colour (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

### TREES B2. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Price? · [8+, 3−] · Red · [3+, 2−] · Green · [5+, 1−]

```
[Diagram as text] Labels: Student? · [8+, 3−] · a · [3+, 0−] · b · [4+, 2−] · c · [1+, 1−]
```

Two candidate splits of [8+, 3−] are shown: Price and Student. Give IG(Price), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Price) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Price) = 0.049; IG(Student) = 0.163; Chosen attribute = Student

**Working:**

- H(parent [8+, 3−]) = −(8/11)·$\log_2$(8/11) = 0.3341 + −(3/11)·$\log_2$(3/11) = 0.5112 = 0.8454
- H(Red [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- H(Green [5+, 1−]) = −(5/6)·$\log_2$(5/6) = 0.2192 + −(1/6)·$\log_2$(1/6) = 0.4308 = 0.6500

- Weighted child entropy = (5/11)·0.9710 + (6/11)·0.6500 = 0.7959
- IG(Price) = 0.8454 − 0.7959 = 0.0495
- H(Student=a [3+, 0−]) = −(3/3)·$\log_2$(3/3) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(Student=b [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Student=c [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- Weighted child entropy (Student) = 0.6827 → IG(Student) = 0.8454 − 0.6827 = 0.1626
- Highest IG → split on Student.
- Gini check: weighted Gini Price = 0.3697, Student = 0.3333 → Gini prefers Student (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B3. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Student? · [5+, 6−] · Some · [1+, 2−] · Full · [2+, 1−] · None · [2+, 3−]

```
[Diagram as text] Labels: Patrons? · [5+, 6−] · a · [2+, 5−] · b · [3+, 1−]
```

Two candidate splits of [5+, 6−] are shown: Student and Patrons. Give IG(Student), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Student) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

**Answer:** IG(Student) = 0.052; IG(Patrons) = 0.150; Chosen attribute = Patrons

**Working:**

- H(parent [5+, 6−]) = −(5/11)·$\log_2$(5/11) = 0.5170 + −(6/11)·$\log_2$(6/11) = 0.4770 = 0.9940
- H(Some [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Full [2+, 1−]) = −(2/3)·$\log_2$(2/3) = 0.3900 + −(1/3)·$\log_2$(1/3) = 0.5283 = 0.9183
- H(None [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy = (3/11)·0.9183 + (3/11)·0.9183 + (5/11)·0.9710 = 0.9422
- IG(Student) = 0.9940 − 0.9422 = 0.0518
- H(Patrons=a [2+, 5−]) = −(2/7)·$\log_2$(2/7) = 0.5164 + −(5/7)·$\log_2$(5/7) = 0.3467 = 0.8631
- H(Patrons=b [3+, 1−]) = −(3/4)·$\log_2$(3/4) = 0.3113 + −(1/4)·$\log_2$(1/4) = 0.5000 = 0.8113
- Weighted child entropy (Patrons) = 0.8443 → IG(Patrons) = 0.9940 − 0.8443 = 0.1498
- Highest IG → split on Patrons.
- Gini check: weighted Gini Student = 0.4606, Patrons = 0.3961 → Gini prefers Patrons (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B4. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [6+, 5−] · Sunny · [1+, 0−] · Overcast · [3+, 2−] · Rain · [2+, 3−]

```
[Diagram as text] Labels: Income? · [6+, 5−] · a · [4+, 2−] · b · [2+, 3−]
```

Two candidate splits of [6+, 5−] are shown: Colour and Income. Give IG(Colour), IG(Income) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Income) = ____

(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.111; IG(Income) = 0.052; Chosen attribute = Colour

**Working:**

- H(parent [6+, 5−]) = −(6/11)·$\log_2$(6/11) = 0.4770 + −(5/11)·$\log_2$(5/11) = 0.5170 = 0.9940
- H(Sunny [1+, 0−]) = −(1/1)·$\log_2$(1/1) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(Overcast [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- H(Rain [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy = (1/11)·0.0000 + (5/11)·0.9710 + (5/11)·0.9710 = 0.8827
- IG(Colour) = 0.9940 − 0.8827 = 0.1113
- H(Income=a [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Income=b [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy (Income) = 0.9422 → IG(Income) = 0.9940 − 0.9422 = 0.0518
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.4364, Income = 0.4606 → Gini prefers Colour (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B5. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Hungry? · [8+, 7−] · Small · [1+, 4−] · Large · [7+, 3−]

```
[Diagram as text] Labels: Patrons? · [8+, 7−] · a · [2+, 3−] · b · [4+, 2−] · c · [2+, 2−]
```

Two candidate splits of [8+, 7−] are shown: Hungry and Patrons. Give IG(Hungry), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Hungry) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

**Answer:** IG(Hungry) = 0.169; IG(Patrons) = 0.039; Chosen attribute = Hungry

**Working:**

- H(parent [8+, 7−]) = −(8/15)·$\log_2$(8/15) = 0.4837 + −(7/15)·$\log_2$(7/15) = 0.5131 = 0.9968
- H(Small [1+, 4−]) = −(1/5)·$\log_2$(1/5) = 0.4644 + −(4/5)·$\log_2$(4/5) = 0.2575 = 0.7219
- H(Large [7+, 3−]) = −(7/10)·$\log_2$(7/10) = 0.3602 + −(3/10)·$\log_2$(3/10) = 0.5211 = 0.8813
- Weighted child entropy = (5/15)·0.7219 + (10/15)·0.8813 = 0.8282
- IG(Hungry) = 0.9968 − 0.8282 = 0.1686
- H(Patrons=a [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Patrons=b [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Patrons=c [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Patrons) = 0.9576 → IG(Patrons) = 0.9968 − 0.9576 = 0.0392
- Highest IG → split on Hungry.
- Gini check: weighted Gini Hungry = 0.3867, Patrons = 0.4711 → Gini prefers Hungry (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B6. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Income? · [3+, 7−] · v1 · [0+, 2−] · v2 · [1+, 3−] · v3 · [2+, 2−]

```
[Diagram as text] Labels: Price? · [3+, 7−] · a · [1+, 2−] · b · [2+, 5−]
```

Two candidate splits of [3+, 7−] are shown: Income and Price. Give IG(Income), IG(Price) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Income) = ____
(2) IG(Price) = ____
(3) Chosen attribute = ____

**Answer:** IG(Income) = 0.157; IG(Price) = 0.002; Chosen attribute = Income

**Working:**

- H(parent [3+, 7−]) = −(3/10)·$\log_2$(3/10) = 0.5211 + −(7/10)·$\log_2$(7/10) = 0.3602 = 0.8813
- H(v1 [0+, 2−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(2/2)·$\log_2$(2/2) = 0.0000 = 0.0000
- H(v2 [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(v3 [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy = (2/10)·0.0000 + (4/10)·0.8113 + (4/10)·1.0000 = 0.7245
- IG(Income) = 0.8813 − 0.7245 = 0.1568
- H(Price=a [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Price=b [2+, 5−]) = −(2/7)·$\log_2$(2/7) = 0.5164 + −(5/7)·$\log_2$(5/7) = 0.3467 = 0.8631
- Weighted child entropy (Price) = 0.8797 → IG(Price) = 0.8813 − 0.8797 = 0.0016
- Highest IG → split on Income.
- Gini check: weighted Gini Income = 0.3500, Price = 0.4190 → Gini prefers Income (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

### TREES B7. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Price? · [8+, 6−] · Red · [2+, 2−] · Green · [6+, 4−]

```
[Diagram as text] Labels: Colour? · [8+, 6−] · a · [7+, 0−] · b · [0+, 4−] · c · [1+, 2−]
```

Two candidate splits of [8+, 6−] are shown: Price and Colour. Give IG(Price), IG(Colour) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Price) = ____
(2) IG(Colour) = ____
(3) Chosen attribute = ____

**Answer:** IG(Price) = 0.006; IG(Colour) = 0.788; Chosen attribute = Colour

**Working:**

- H(parent [8+, 6−]) = −(8/14)·$\log_2$(8/14) = 0.4613 + −(6/14)·$\log_2$(6/14) = 0.5239 = 0.9852
- H(Red [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- H(Green [6+, 4−]) = −(6/10)·$\log_2$(6/10) = 0.4422 + −(4/10)·$\log_2$(4/10) = 0.5288 = 0.9710
- Weighted child entropy = (4/14)·1.0000 + (10/14)·0.9710 = 0.9793
- IG(Price) = 0.9852 − 0.9793 = 0.0060
- H(Colour=a [7+, 0−]) = −(7/7)·$\log_2$(7/7) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(Colour=b [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- H(Colour=c [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- Weighted child entropy (Colour) = 0.1968 → IG(Colour) = 0.9852 − 0.1968 = 0.7885
- Highest IG → split on Colour.
- Gini check: weighted Gini Price = 0.4857, Colour = 0.0952 → Gini prefers Colour (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B8. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Outlook? · [6+, 3−] · Red · [4+, 3−] · Green · [2+, 0−]

```
[Diagram as text] Labels: Price? · [6+, 3−] · a · [3+, 0−] · b · [1+, 1−] · c · [2+, 2−]
```

Two candidate splits of [6+, 3−] are shown: Outlook and Price. Give IG(Outlook), IG(Price) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Outlook) = ____
(2) IG(Price) = ____
(3) Chosen attribute = ____

**Answer:** IG(Outlook) = 0.152; IG(Price) = 0.252; Chosen attribute = Price

**Working:**

- H(parent [6+, 3−]) = −(6/9)·$\log_2$(6/9) = 0.3900 + −(3/9)·$\log_2$(3/9) = 0.5283 = 0.9183
- H(Red [4+, 3−]) = −(4/7)·$\log_2$(4/7) = 0.4613 + −(3/7)·$\log_2$(3/7) = 0.5239 = 0.9852
- H(Green [2+, 0−]) = −(2/2)·$\log_2$(2/2) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- Weighted child entropy = (7/9)·0.9852 + (2/9)·0.0000 = 0.7663
- IG(Outlook) = 0.9183 − 0.7663 = 0.1520
- H(Price=a [3+, 0−]) = −(3/3)·$\log_2$(3/3) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(Price=b [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- H(Price=c [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Price) = 0.6667 → IG(Price) = 0.9183 − 0.6667 = 0.2516
- Highest IG → split on Price.
- Gini check: weighted Gini Outlook = 0.3810, Price = 0.3333 → Gini prefers Price (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B9. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [3+, 7−] · Sunny · [2+, 3−] · Overcast · [1+, 4−]

```
[Diagram as text] Labels: Income? · [3+, 7−] · a · [0+, 2−] · b · [1+, 4−] · c · [2+, 1−]
```

Two candidate splits of [3+, 7−] are shown: Colour and Income. Give IG(Colour), IG(Income) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Income) = ____
(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.035; IG(Income) = 0.245; Chosen attribute = Income

**Working:**

- H(parent [3+, 7−]) = −(3/10)·$\log_2$(3/10) = 0.5211 + −(7/10)·$\log_2$(7/10) = 0.3602 = 0.8813
- H(Sunny [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Overcast [1+, 4−]) = −(1/5)·$\log_2$(1/5) = 0.4644 + −(4/5)·$\log_2$(4/5) = 0.2575 = 0.7219
- Weighted child entropy = (5/10)·0.9710 + (5/10)·0.7219 = 0.8464
- IG(Colour) = 0.8813 − 0.8464 = 0.0349

- H(Income=a [0+, 2−]) = 0 (pure part: $0 \cdot \log_2 0 = 0$) + $-(2/2) \cdot \log_2(2/2)$ = 0.0000 = 0.0000
- H(Income=b [1+, 4−]) = $-(1/5) \cdot \log_2(1/5)$ = 0.4644 + $-(4/5) \cdot \log_2(4/5)$ = 0.2575 = 0.7219
- H(Income=c [2+, 1−]) = $-(2/3) \cdot \log_2(2/3)$ = 0.3900 + $-(1/3) \cdot \log_2(1/3)$ = 0.5283 = 0.9183
- Weighted child entropy (Income) = 0.6365 → IG(Income) = 0.8813 − 0.6365 = 0.2448
- Highest IG → split on Income.
- Gini check: weighted Gini Colour = 0.4000, Income = 0.2933 → Gini prefers Income (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B10. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Size? · [6+, 8−] · Small · [2+, 3−] · Large · [4+, 5−]

```
[Diagram as text] Labels: Patrons? · [6+, 8−] · a · [0+, 3−] · b · [4+, 2−] · c · [2+, 3−]
```

Two candidate splits of [6+, 8−] are shown: Size and Patrons. Give IG(Size), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Size) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

**Answer:** IG(Size) = 0.001; IG(Patrons) = 0.245; Chosen attribute = Patrons

**Working:**

- H(parent [6+, 8−]) = $-(6/14) \cdot \log_2(6/14)$ = 0.5239 + $-(8/14) \cdot \log_2(8/14)$ = 0.4613 = 0.9852
- H(Small [2+, 3−]) = $-(2/5) \cdot \log_2(2/5)$ = 0.5288 + $-(3/5) \cdot \log_2(3/5)$ = 0.4422 = 0.9710
- H(Large [4+, 5−]) = $-(4/9) \cdot \log_2(4/9)$ = 0.5200 + $-(5/9) \cdot \log_2(5/9)$ = 0.4711 = 0.9911
- Weighted child entropy = $(5/14) \cdot 0.9710 + (9/14) \cdot 0.9911$ = 0.9839
- IG(Size) = 0.9852 − 0.9839 = 0.0013
- H(Patrons=a [0+, 3−]) = 0 (pure part: $0 \cdot \log_2 0 = 0$) + $-(3/3) \cdot \log_2(3/3)$ = 0.0000 = 0.0000
- H(Patrons=b [4+, 2−]) = $-(4/6) \cdot \log_2(4/6)$ = 0.3900 + $-(2/6) \cdot \log_2(2/6)$ = 0.5283 = 0.9183
- H(Patrons=c [2+, 3−]) = $-(2/5) \cdot \log_2(2/5)$ = 0.5288 + $-(3/5) \cdot \log_2(3/5)$ = 0.4422 = 0.9710
- Weighted child entropy (Patrons) = 0.7403 → IG(Patrons) = 0.9852 − 0.7403 = 0.2449
- Highest IG → split on Patrons.
- Gini check: weighted Gini Size = 0.4889, Patrons = 0.3619 → Gini prefers Patrons (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B11. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Patrons? · [6+, 5−] · Red · [4+, 2−] · Green · [2+, 3−]

```
[Diagram as text] Labels: Student? · [6+, 5−] · a · [3+, 1−] · b · [1+, 2−] · c · [2+, 2−]
```

Two candidate splits of [6+, 5−] are shown: Patrons and Student. Give IG(Patrons), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Patrons) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Patrons) = 0.052; IG(Student) = 0.085; Chosen attribute = Student

**Working:**

- H(parent [6+, 5−]) = −(6/11)·log₂(6/11) = 0.4770 + −(5/11)·log₂(5/11) = 0.5170 = 0.9940
- H(Red [4+, 2−]) = −(4/6)·log₂(4/6) = 0.3900 + −(2/6)·log₂(2/6) = 0.5283 = 0.9183
- H(Green [2+, 3−]) = −(2/5)·log₂(2/5) = 0.5288 + −(3/5)·log₂(3/5) = 0.4422 = 0.9710
- Weighted child entropy = (6/11)·0.9183 + (5/11)·0.9710 = 0.9422
- IG(Patrons) = 0.9940 − 0.9422 = 0.0518
- H(Student=a [3+, 1−]) = −(3/4)·log₂(3/4) = 0.3113 + −(1/4)·log₂(1/4) = 0.5000 = 0.8113
- H(Student=b [1+, 2−]) = −(1/3)·log₂(1/3) = 0.5283 + −(2/3)·log₂(2/3) = 0.3900 = 0.9183
- H(Student=c [2+, 2−]) = −(2/4)·log₂(2/4) = 0.5000 + −(2/4)·log₂(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Student) = 0.9091 → IG(Student) = 0.9940 − 0.9091 = 0.0849
- Highest IG → split on Student.
- Gini check: weighted Gini Patrons = 0.4606, Student = 0.4394 → Gini prefers Student (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B12. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [5+, 8−] · Sunny · [1+, 2−] · Overcast · [1+, 4−] · Rain · [3+, 2−]

```
[Diagram as text] Labels: Student? · [5+, 8−] · a · [1+, 3−] · b · [4+, 5−]
```

Two candidate splits of [5+, 8−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.098; IG(Student) = 0.025; Chosen attribute = Colour

**Working:**

- H(parent [5+, 8−]) = −(5/13)·log₂(5/13) = 0.5302 + −(8/13)·log₂(8/13) = 0.4310 = 0.9612
- H(Sunny [1+, 2−]) = −(1/3)·log₂(1/3) = 0.5283 + −(2/3)·log₂(2/3) = 0.3900 = 0.9183
- H(Overcast [1+, 4−]) = −(1/5)·log₂(1/5) = 0.4644 + −(4/5)·log₂(4/5) = 0.2575 = 0.7219
- H(Rain [3+, 2−]) = −(3/5)·log₂(3/5) = 0.4422 + −(2/5)·log₂(2/5) = 0.5288 = 0.9710
- Weighted child entropy = (3/13)·0.9183 + (5/13)·0.7219 + (5/13)·0.9710 = 0.8630
- IG(Colour) = 0.9612 − 0.8630 = 0.0982
- H(Student=a [1+, 3−]) = −(1/4)·log₂(1/4) = 0.5000 + −(3/4)·log₂(3/4) = 0.3113 = 0.8113
- H(Student=b [4+, 5−]) = −(4/9)·log₂(4/9) = 0.5200 + −(5/9)·log₂(5/9) = 0.4711 = 0.9911
- Weighted child entropy (Student) = 0.9358 → IG(Student) = 0.9612 − 0.9358 = 0.0255
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.4103, Student = 0.4573 → Gini prefers Colour (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B13. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [8+, 7−] · Yes · [0+, 4−] · No · [8+, 3−]

```
[Diagram as text] Labels: Student? · [8+, 7−] · a · [3+, 2−] · b · [5+, 1−] · c · [0+, 4−]
```

Two candidate splits of [8+, 7−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.377; IG(Student) = 0.413; Chosen attribute = Student

**Working:**

- H(parent [8+, 7−]) = −(8/15)·$\log_2$(8/15) = 0.4837 + −(7/15)·$\log_2$(7/15) = 0.5131 = 0.9968
- H(Yes [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- H(No [8+, 3−]) = −(8/11)·$\log_2$(8/11) = 0.3341 + −(3/11)·$\log_2$(3/11) = 0.5112 = 0.8454
- Weighted child entropy = (4/15)·0.0000 + (11/15)·0.8454 = 0.6199
- IG(Colour) = 0.9968 − 0.6199 = 0.3769
- H(Student=a [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- H(Student=b [5+, 1−]) = −(5/6)·$\log_2$(5/6) = 0.2192 + −(1/6)·$\log_2$(1/6) = 0.4308 = 0.6500
- H(Student=c [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- Weighted child entropy (Student) = 0.5837 → IG(Student) = 0.9968 − 0.5837 = 0.4131
- Highest IG → split on Student.
- Gini check: weighted Gini Colour = 0.2909, Student = 0.2711 → Gini prefers Student (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B14. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Size? · [4+, 6−] · Sunny · [3+, 3−] · Overcast · [1+, 3−]

```
[Diagram as text] Labels: Patrons? · [4+, 6−] · a · [1+, 1−] · b · [1+, 2−] · c · [2+, 3−]
```

Two candidate splits of [4+, 6−] are shown: Size and Patrons. Give IG(Size), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Size) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

**Answer:** IG(Size) = 0.046; IG(Patrons) = 0.010; Chosen attribute = Size

**Working:**

- H(parent [4+, 6−]) = −(4/10)·$\log_2$(4/10) = 0.5288 + −(6/10)·$\log_2$(6/10) = 0.4422 = 0.9710
- H(Sunny [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- H(Overcast [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- Weighted child entropy = (6/10)·1.0000 + (4/10)·0.8113 = 0.9245
- IG(Size) = 0.9710 − 0.9245 = 0.0464
- H(Patrons=a [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- H(Patrons=b [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Patrons=c [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy (Patrons) = 0.9610 → IG(Patrons) = 0.9710 − 0.9610 = 0.0100
- Highest IG → split on Size.
- Gini check: weighted Gini Size = 0.4500, Patrons = 0.4733 → Gini prefers Size (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B15. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [8+, 6−] · Low · [1+, 4−] · Medium · [4+, 0−] · High · [3+, 2−]

```
[Diagram as text] Labels: Student? · [8+, 6−] · a · [3+, 3−] · b · [5+, 3−]
```

Two candidate splits of [8+, 6−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

**Answer:** IG(Colour) = 0.381; IG(Student) = 0.011; Chosen attribute = Colour

**Working:**

- H(parent [8+, 6−]) = −(8/14)·$\log_2$(8/14) = 0.4613 + −(6/14)·$\log_2$(6/14) = 0.5239 = 0.9852
- H(Low [1+, 4−]) = −(1/5)·$\log_2$(1/5) = 0.4644 + −(4/5)·$\log_2$(4/5) = 0.2575 = 0.7219
- H(Medium [4+, 0−]) = −(4/4)·$\log_2$(4/4) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(High [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- Weighted child entropy = (5/14)·0.7219 + (4/14)·0.0000 + (5/14)·0.9710 = 0.6046
- IG(Colour) = 0.9852 − 0.6046 = 0.3806
- H(Student=a [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- H(Student=b [5+, 3−]) = −(5/8)·$\log_2$(5/8) = 0.4238 + −(3/8)·$\log_2$(3/8) = 0.5306 = 0.9544
- Weighted child entropy (Student) = 0.9740 → IG(Student) = 0.9852 − 0.9740 = 0.0113
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.2857, Student = 0.4821 → Gini prefers Colour (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

## TREES B16. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Outlook? · [3+, 4−] · v1 · [1+, 3−] · v2 · [1+, 0−] · v3 · [1+, 1−]

```
[Diagram as text] Labels: Hungry? · [3+, 4−] · a · [2+, 3−] · b · [1+, 1−]
```

Two candidate splits of [3+, 4−] are shown: Outlook and Hungry. Give IG(Outlook), IG(Hungry) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Outlook) = ____
(2) IG(Hungry) = ____
(3) Chosen attribute = ____

**Answer:** IG(Outlook) = 0.236; IG(Hungry) = 0.006; Chosen attribute = Outlook

**Working:**

- H(parent [3+, 4−]) = −(3/7)·$\log_2$(3/7) = 0.5239 + −(4/7)·$\log_2$(4/7) = 0.4613 = 0.9852
- H(v1 [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(v2 [1+, 0−]) = −(1/1)·$\log_2$(1/1) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(v3 [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- Weighted child entropy = (4/7)·0.8113 + (1/7)·0.0000 + (2/7)·1.0000 = 0.7493
- IG(Outlook) = 0.9852 − 0.7493 = 0.2359
- H(Hungry=a [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Hungry=b [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000

- Weighted child entropy (Hungry) = 0.9793 → IG(Hungry) = 0.9852 − 0.9793 = 0.0060
- Highest IG → split on Outlook.
- Gini check: weighted Gini Outlook = 0.3571, Hungry = 0.4857 → Gini prefers Outlook (same choice).

**Takeaway:** IG: pick the highest. Weighted Gini: pick the lowest.

*Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'*

---

# 🏰 MDPs & Reinforcement Learning (Workshop 6)

## Part B: calculation drills (16)

### MDP B1. [BLANK · Exam+ · calculation]

| U=0.53 |
U=0.83 | s (U=0.44) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.5110

**Working:**

- Up: 0.8×(−0.04 + 0.53)
- Left slip: 0.1×(−0.04 + 0.83)
- Right slip: 0.1×(−0.04 + 0.44) (wall: stays in s)
- Q = 0.5110.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B2. [BLANK · Exam+ · calculation]

| U=0.49 |
U=0.41 | s (U=0.75) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.4680

**Working:**

- Up: 0.8×(−0.04 + 0.49)
- Left slip: 0.1×(−0.04 + 0.41)
- Right slip: 0.1×(−0.04 + 0.75) (wall: stays in s)
- Q = 0.4680.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B3. [BLANK · Exam+ · calculation]

| U=0.46 |
wall | s (U=0.43) | U=0.71

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.4420

**Working:**

- Up: 0.8×(−0.04 + 0.46)

- Left slip: 0.1×(−0.04 + 0.43) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.71)
- Q = 0.4420.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B4. [BLANK · Exam+ · calculation]

| U=0.51 |
U=0.36 | s (U=0.51) | U=0.25

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.4290

**Working:**

- Up: 0.8×(−0.04 + 0.51)
- Left slip: 0.1×(−0.04 + 0.36)
- Right slip: 0.1×(−0.04 + 0.25)
- Q = 0.4290.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B5. [BLANK · Exam+ · calculation]

| U=0.37 |
wall | s (U=0.6) | U=0.21

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.3370

**Working:**

- Up: 0.8×(−0.04 + 0.37)
- Left slip: 0.1×(−0.04 + 0.6) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.21)
- Q = 0.3370.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B6. [BLANK · Exam+ · calculation]

| U=0.3 |
U=0.5 | s (U=0.52) | U=0.5

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.3000

**Working:**

- Up: 0.8×(−0.04 + 0.3)
- Left slip: 0.1×(−0.04 + 0.5)
- Right slip: 0.1×(−0.04 + 0.5)
- Q = 0.3000.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B7. [BLANK · Exam+ · calculation]

| U=0.45 |
U=0.74 | s (U=0.35) | U=0.47

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.4410

**Working:**

- Up: 0.8×(−0.04 + 0.45)
- Left slip: 0.1×(−0.04 + 0.74)
- Right slip: 0.1×(−0.04 + 0.47)
- Q = 0.4410.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B8. [BLANK · Exam+ · calculation]

| U=0.57 |
wall | s (U=0.79) | U=0.65

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.5600

**Working:**

- Up: 0.8×(−0.04 + 0.57)
- Left slip: 0.1×(−0.04 + 0.79) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.65)
- Q = 0.5600.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B9. [BLANK · Exam+ · calculation]

| U=0.3 |
wall | s (U=0.14) | U=0.22

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.2360

**Working:**

- Up: 0.8×(−0.04 + 0.3)
- Left slip: 0.1×(−0.04 + 0.14) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.22)
- Q = 0.2360.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B10. [BLANK · Exam+ · calculation]

| U=0.73 |
wall | s (U=0.56) | U=0.31

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.6310

**Working:**

- Up: 0.8×(−0.04 + 0.73)
- Left slip: 0.1×(−0.04 + 0.56) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.31)
- Q = 0.6310.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B11. [BLANK · Exam+ · calculation]

| U=0.13 |
U=0.65 | s (U=0.7) | U=0.63

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.1920

**Working:**

- Up: 0.8×(−0.04 + 0.13)
- Left slip: 0.1×(−0.04 + 0.65)
- Right slip: 0.1×(−0.04 + 0.63)
- Q = 0.1920.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B12. [BLANK · Exam+ · calculation]

| U=0.31 |
U=0.44 | s (U=0.48) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.3000

**Working:**

- Up: 0.8×(−0.04 + 0.31)
- Left slip: 0.1×(−0.04 + 0.44)
- Right slip: 0.1×(−0.04 + 0.48) (wall: stays in s)
- Q = 0.3000.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B13. [BLANK · Exam+ · calculation]

| U=0.74 |
U=0.36 | s (U=0.46) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.6340

**Working:**

- Up: 0.8×(−0.04 + 0.74)
- Left slip: 0.1×(−0.04 + 0.36)
- Right slip: 0.1×(−0.04 + 0.46) (wall: stays in s)
- Q = 0.6340.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B14. [BLANK · Exam+ · calculation]

| U=0.41 |
U=0.13 | s (U=0.2) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.3210

**Working:**

- Up: 0.8×(−0.04 + 0.41)
- Left slip: 0.1×(−0.04 + 0.13)
- Right slip: 0.1×(−0.04 + 0.2) (wall: stays in s)
- Q = 0.3210.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B15. [BLANK · Exam+ · calculation]

| U=0.27 |
U=0.13 | s (U=0.74) | U=0.62

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the

current utilities shown, compute Q(s, Up).

**Answer:** 0.2510

**Working:**

- Up: 0.8×(−0.04 + 0.27)
- Left slip: 0.1×(−0.04 + 0.13)
- Right slip: 0.1×(−0.04 + 0.62)
- Q = 0.2510.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

### MDP B16. [BLANK · Exam+ · calculation]

| U=0.67 |
wall | s (U=0.88) | U=0.5

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

**Answer:** 0.6340

**Working:**

- Up: 0.8×(−0.04 + 0.67)
- Left slip: 0.1×(−0.04 + 0.88) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.5)
- Q = 0.6340.

**Takeaway:** Slips go perpendicular to the intended move; bumping a wall means staying put.

*Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)*

---

# 🎲 Probability & Bayesian Networks (Lecture 11)

## Part B: calculation drills (16)

### BAYES B1. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, ¬m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 2.568e-4; s_¬B = 4.980e-2; P(b|evidence) = 0.0051

**Working:**

- s_B = 0.001×{P(e)[0.95·0.9·0.3 + 0.05·0.05·0.99] + P(¬e)[0.94·0.9·0.3 + 0.06·0.05·0.99]} = 2.56774e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.3 + 0.71·0.05·0.99] + P(¬e)[0.001·0.9·0.3 + 0.999·0.05·0.99]} = 4.97981e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0051.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B2. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.9
f | t | 0.29
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.8
f | 0.02 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 4.090e-3; s_¬B = 4.405e-2; P(b|evidence) = 0.0850

**Working:**

- s_B = 0.1×{P(e)[0.9·0.05·0.8 + 0.1·0.98·0.05] + P(¬e)[0.9·0.05·0.8 + 0.1·0.98·0.05]} = 4.09000e-3.
- s_¬B = 0.9×{P(e)[0.29·0.05·0.8 + 0.71·0.98·0.05] + P(¬e)[0.005·0.05·0.8 + 0.995·0.98·0.05]} = 4.40549e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0850.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B3. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 6.637e-5; s_¬B = 9.586e-3; P(b|evidence) = 0.0069

**Working:**

- s_B = 0.001×{P(e)[0.95·0.1·0.7 + 0.05·0.95·0.01] + P(¬e)[0.94·0.1·0.7 + 0.06·0.95·0.01]} = 6.63712e-5.
- s_¬B = 0.999×{P(e)[0.29·0.1·0.7 + 0.71·0.95·0.01] + P(¬e)[0.001·0.1·0.7 + 0.999·0.95·0.01]} = 9.58587e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0069.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B4. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.05 | 0.02
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 2.437e-3; s_¬B = 1.881e-2; P(b|evidence) = 0.1147

**Working:**

- s_B = 0.02×{P(e)[0.95·0.2·0.7 + 0.05·0.95·0.02] + P(¬e)[0.85·0.2·0.7 + 0.15·0.95·0.02]} = 2.43748e-3.
- s_¬B = 0.98×{P(e)[0.29·0.2·0.7 + 0.71·0.95·0.02] + P(¬e)[0.001·0.2·0.7 + 0.999·0.95·0.02]} = 1.88071e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1147.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B5. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.94
f | t | 0.4
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.02 | 0.01
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 1.727e-3; s_¬B = 9.409e-1; P(b|evidence) = 0.0018

**Working:**

- s_B = 0.02×{P(e)[0.98·0.1·0.3 + 0.02·0.98·0.99] + P(¬e)[0.94·0.1·0.3 + 0.06·0.98·0.99]} = 1.72674e-3.
- s_¬B = 0.98×{P(e)[0.4·0.1·0.3 + 0.6·0.98·0.99] + P(¬e)[0.01·0.1·0.3 + 0.99·0.98·0.99]} = 9.40863e-1.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0018.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B6. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.01 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.3
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.1 | 0.02
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 1.582e-3; s_¬B = 8.605e-1; P(b|evidence) = 0.0018

**Working:**

- s_B = 0.01×{P(e)[0.95·0.2·0.4 + 0.05·0.9·0.98] + P(¬e)[0.9·0.2·0.4 + 0.1·0.9·0.98]} = 1.58195e-3.
- s_¬B = 0.99×{P(e)[0.3·0.2·0.4 + 0.7·0.9·0.98] + P(¬e)[0.001·0.2·0.4 + 0.999·0.9·0.98]} = 8.60516e-1.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0018.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B7. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.6
f | 0.05 | 0.02
```

Compute P(B | j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.078e-2; s_¬B = 4.189e-3; P(b|evidence) = 0.9238

**Working:**

- s_B = 0.1×{P(e)[0.95·0.9·0.6 + 0.05·0.05·0.02] + P(¬e)[0.94·0.9·0.6 + 0.06·0.05·0.02]} = 5.07768e-2.
- s_¬B = 0.9×{P(e)[0.29·0.9·0.6 + 0.71·0.05·0.02] + P(¬e)[0.001·0.9·0.6 + 0.999·0.05·0.02]} = 4.18898e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.9238.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B8. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.4
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.8
f | 0.02 | 0.05
```

Compute P(B | j, ¬m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 3.120e-3; s_¬B = 2.003e-2; P(b|evidence) = 0.1348

**Working:**

- s_B = 0.02×{P(e)[0.95·0.9·0.2 + 0.05·0.02·0.95] + P(¬e)[0.85·0.9·0.2 + 0.15·0.02·0.95]} = 3.12022e-3.
- s_¬B = 0.98×{P(e)[0.4·0.9·0.2 + 0.6·0.02·0.95] + P(¬e)[0.005·0.9·0.2 + 0.995·0.02·0.95]} = 2.00321e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1348.

- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B9. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.1 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 3.090e-3; s_¬B = 4.041e-2; P(b|evidence) = 0.0710

**Working:**

- s_B = 0.1×{P(e)[0.95·0.05·0.6 + 0.05·0.9·0.05] + P(¬e)[0.94·0.05·0.6 + 0.06·0.9·0.05]} = 3.08970e-3.
- s_¬B = 0.9×{P(e)[0.29·0.05·0.6 + 0.71·0.9·0.05] + P(¬e)[0.001·0.05·0.6 + 0.999·0.9·0.05]} = 4.04085e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0710.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B10. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

**Working:**

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B11. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.02 | 0.02
```

Compute P(B | j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.267e-2; s_¬B = 1.270e-2; P(b|evidence) = 0.8057

**Working:**

- s_B = 0.1×{P(e)[0.95·0.8·0.7 + 0.05·0.02·0.02] + P(¬e)[0.94·0.8·0.7 + 0.06·0.02·0.02]} = 5.26704e-2.
- s_¬B = 0.9×{P(e)[0.3·0.8·0.7 + 0.7·0.02·0.02] + P(¬e)[0.01·0.8·0.7 + 0.99·0.02·0.02]} = 1.26992e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.8057.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B12. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.85
f | t | 0.29
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.1 | 0.01
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 2.016e-4; s_¬B = 8.856e-1; P(b|evidence) = 0.0002

**Working:**

- s_B = 0.001×{P(e)[0.9·0.2·0.4 + 0.1·0.9·0.99] + P(¬e)[0.85·0.2·0.4 + 0.15·0.9·0.99]} = 2.01569e-4.
- s_¬B = 0.999×{P(e)[0.29·0.2·0.4 + 0.71·0.9·0.99] + P(¬e)[0.005·0.2·0.4 + 0.995·0.9·0.99]} = 8.85596e-1.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0002.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B13. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.05 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.2
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.8
f | 0.02 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 7.448e-3; s_¬B = 4.728e-2; P(b|evidence) = 0.1361

**Working:**

- s_B = 0.05×{P(e)[0.95·0.2·0.8 + 0.05·0.98·0.05] + P(¬e)[0.9·0.2·0.8 + 0.1·0.98·0.05]} = 7.44778e-3.
- s_¬B = 0.95×{P(e)[0.2·0.2·0.8 + 0.8·0.98·0.05] + P(¬e)[0.005·0.2·0.8 + 0.995·0.98·0.05]} = 4.72829e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1361.

- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B14. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)$\Sigma_e$ P(e)$\Sigma_a$ P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

**Working:**

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

## BAYES B15. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.1 | 0.02
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ$_e$ P(e)Σ$_a$ P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.760e-4; s_¬B = 1.766e-2; P(b|evidence) = 0.0316

**Working:**

- s_B = 0.02×{P(e)[0.98·0.05·0.6 + 0.02·0.9·0.02] + P(¬e)[0.9·0.05·0.6 + 0.1·0.9·0.02]} = 5.76038e-4.
- s_¬B = 0.98×{P(e)[0.4·0.05·0.6 + 0.6·0.9·0.02] + P(¬e)[0.001·0.05·0.6 + 0.999·0.9·0.02]} = 1.76611e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0316.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

### BAYES B16. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ$_e$ P(e)Σ$_a$ P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

**Answer:** s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

**Working:**

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

**Takeaway:** Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

*Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises*

---

# CNNs & Computer Vision (Lecture 7)

## Part B: calculation drills (16)

### CNN B1. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B2. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 14; flatten length = 1568; total params = 18106

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/1 \rfloor + 1 = 28$ → 8×28×28.
- Pool: $\lfloor 28/2 \rfloor = 14$.
- Conv2 (padding 1, stride 1): 14; Pool: 7 → 32×7×7.
- Flatten: 32×7×7 = 1568.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (1568+1)×10 = 15690; total 18106.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B3. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 16; flatten length = 1024; total params = 11498

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/2 \rfloor + 1 = 32$ → 8×32×32.
- Pool: $\lfloor 32/2 \rfloor = 16$.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 16×8×8.
- Flatten: 16×8×8 = 1024.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×16 = 1168; linear (1024+1)×10 = 10250; total 11498.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

## CNN B4. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 32; flatten length = 8192; total params = 84346

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64$ → 8×64×64.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: 16 → 32×16×16.
- Flatten: 32×16×16 = 8192.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (8192+1)×10 = 81930; total 84346.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

## CNN B5. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 32; flatten length = 4096; total params = 43450

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64$ → 16×64×64.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: 16 → 16×16×16.

- Flatten: 16×16×16 = 4096.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×16 = 2320; linear (4096+1)×10 = 40970; total 43450.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B6. [BLANKS · Exam+ · calculation]

Input 1×32×32. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 16; flatten length = 1024; total params = 12730

**Working:**

- Conv1: ⌊(32 + 2 − 3)/1⌋ + 1 = 32 → 16×32×32.
- Pool: ⌊32/2⌋ = 16.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 16×8×8.
- Flatten: 16×8×8 = 1024.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×16 = 2320; linear (1024+1)×10 = 10250; total 12730.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B7. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: ⌊(28 + 2 − 3)/2⌋ + 1 = 14 → 8×14×14.
- Pool: ⌊14/2⌋ = 7.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B8. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 16; flatten length = 2048; total params = 25290

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/2 \rfloor + 1 = 32$ → 16×32×32.
- Pool: $\lfloor 32/2 \rfloor = 16$.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 32×8×8.
- Flatten: 32×8×8 = 2048.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×32 = 4640; linear (2048+1)×10 = 20490; total 25290.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B9. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B10. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14 \rightarrow 8\times14\times14$.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: $3 \rightarrow 32\times3\times3$.
- Flatten: $32\times3\times3 = 288$.
- Params: conv1 $(9\times1+1)\times8 = 80$; conv2 $(9\times8+1)\times32 = 2336$; linear $(288+1)\times10 = 2890$; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

## CNN B11. [BLANKS · Exam+ · calculation]

Input 1×32×32. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 8; flatten length = 256; total params = 3818

**Working:**

- Conv1: $\lfloor (32 + 2 - 3)/2 \rfloor + 1 = 16 \rightarrow 8\times16\times16$.
- Pool: $\lfloor 16/2 \rfloor = 8$.
- Conv2 (padding 1, stride 1): 8; Pool: $4 \rightarrow 16\times4\times4$.
- Flatten: $16\times4\times4 = 256$.
- Params: conv1 $(9\times1+1)\times8 = 80$; conv2 $(9\times8+1)\times16 = 1168$; linear $(256+1)\times10 = 2570$; total 3818.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

## CNN B12. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14 \rightarrow 8\times14\times14$.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: $3 \rightarrow 32\times3\times3$.
- Flatten: $32\times3\times3 = 288$.
- Params: conv1 $(9\times1+1)\times8 = 80$; conv2 $(9\times8+1)\times32 = 2336$; linear $(288+1)\times10 = 2890$; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B13. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 16; flatten length = 1024; total params = 12730

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/2 \rfloor + 1 = 32$ → 16×32×32.
- Pool: $\lfloor 32/2 \rfloor = 16$.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 16×8×8.
- Flatten: 16×8×8 = 1024.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×16 = 2320; linear (1024+1)×10 = 10250; total 12730.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B14. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 14; flatten length = 1568; total params = 18106

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/1 \rfloor + 1 = 28$ → 8×28×28.
- Pool: $\lfloor 28/2 \rfloor = 14$.
- Conv2 (padding 1, stride 1): 14; Pool: 7 → 32×7×7.
- Flatten: 32×7×7 = 1568.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (1568+1)×10 = 15690; total 18106.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B15. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 7; flatten length = 288; total params = 5306

**Working:**

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14 \rightarrow 8 \times 14 \times 14$.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: $3 \rightarrow 32 \times 3 \times 3$.
- Flatten: $32 \times 3 \times 3 = 288$.
- Params: conv1 $(9 \times 1 + 1) \times 8 = 80$; conv2 $(9 \times 8 + 1) \times 32 = 2336$; linear $(288 + 1) \times 10 = 2890$; total 5306.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

### CNN B16. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

**Answer:** size after pool 1 = 32; flatten length = 4096; total params = 43450

**Working:**

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64 \rightarrow 16 \times 64 \times 64$.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: $16 \rightarrow 16 \times 16 \times 16$.
- Flatten: $16 \times 16 \times 16 = 4096$.
- Params: conv1 $(9 \times 1 + 1) \times 16 = 160$; conv2 $(9 \times 16 + 1) \times 16 = 2320$; linear $(4096 + 1) \times 10 = 40970$; total 43450.

**Takeaway:** Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

*Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts*

---

# 📦 Object Detection (IoU & NMS) (Lecture 7)

## Part B: calculation drills (16)

### DETECT B1. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (7,6)–(11,11) | 0.58
B2 | (8,2)–(12,9) | 0.74
B3 | (6,6)–(13,9) | 0.95
B4 | (5,2)–(8,7) | 0.82
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B2,B3,B4
(Also accepted: B2, B3, B4, B3,B4,B2)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B4(0.82), B2(0.74), B1(0.58).
- Keep B3.
- IoU(B3, B4) = 0.059 ≤ thr → keep for now
- IoU(B3, B2) = 0.324 ≤ thr → keep for now
- IoU(B3, B1) = 0.414 > thr → suppress B1
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- Keep B2.
- Remaining: B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B2. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (3,7)–(10,10) | 0.74
B2 | (6,5)–(9,11) | 0.82
B3 | (4,6)–(13,8) | 0.66
B4 | (2,2)–(7,6) | 0.9
B5 | (5,4)–(7,8) | 0.58
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B1,B2,B3,B4,B5
(Also accepted: B1, B2, B3, B4, B5, B4,B2,B1,B3,B5)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B2(0.82), B1(0.74), B3(0.66), B5(0.58).
- Keep B4.
- IoU(B4, B2) = 0.027 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B5) = 0.167 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.300 ≤ thr → keep for now
- IoU(B2, B3) = 0.200 ≤ thr → keep for now
- IoU(B2, B5) = 0.130 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B3) = 0.182 ≤ thr → keep for now
- IoU(B1, B5) = 0.074 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B5) = 0.182 ≤ thr → keep for now
- Keep B5.
- Remaining: B1,B2,B3,B4,B5.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B3. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (4,6)–(8,9) | 0.58
B2 | (2,2)–(7,10) | 0.74
B3 | (4,6)–(7,9) | 0.66
B4 | (2,3)–(4,6) | 0.9
B5 | (4,1)–(8,5) | 0.82
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B2,B3,B4,B5
(Also accepted: B2, B3, B4, B5, B4,B5,B2,B3)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B5(0.82), B2(0.74), B3(0.66), B1(0.58).
- Keep B4.
- IoU(B4, B5) = 0.000 ≤ thr → keep for now
- IoU(B4, B2) = 0.150 ≤ thr → keep for now
- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B5.
- IoU(B5, B2) = 0.191 ≤ thr → keep for now
- IoU(B5, B3) = 0.000 ≤ thr → keep for now
- IoU(B5, B1) = 0.000 ≤ thr → keep for now

- Keep B2.
- IoU(B2, B3) = 0.225 ≤ thr → keep for now
- IoU(B2, B1) = 0.209 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B1) = 0.750 > thr → suppress B1
- Remaining: B2,B3,B4,B5.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B4. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (0,4)–(6,10) | 0.82
B2 | (2,8)–(4,10) | 0.95
B3 | (0,8)–(7,12) | 0.74
B4 | (0,5)–(4,9) | 0.58
B5 | (0,5)–(2,9) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

**Answer:** B1,B2,B3,B5
(Also accepted: B1, B2, B3, B5, B2,B5,B1,B3)

**Working:**

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B5(0.9), B1(0.82), B3(0.74), B4(0.58).
- Keep B2.
- IoU(B2, B5) = 0.000 ≤ thr → keep for now
- IoU(B2, B1) = 0.111 ≤ thr → keep for now
- IoU(B2, B3) = 0.143 ≤ thr → keep for now
- IoU(B2, B4) = 0.111 ≤ thr → keep for now
- Keep B5.
- IoU(B5, B1) = 0.222 ≤ thr → keep for now
- IoU(B5, B3) = 0.059 ≤ thr → keep for now
- IoU(B5, B4) = 0.500 > thr → suppress B4
- Keep B1.
- IoU(B1, B3) = 0.231 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B5.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B5. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (2,7)–(5,11) | 0.74
B2 | (1,6)–(3,12) | 0.82
B3 | (2,5)–(6,11) | 0.95
B4 | (3,6)–(5,9) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

**Answer:** B2,B3,B4
(Also accepted: B2, B3, B4, B3,B4,B2)

**Working:**

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B4(0.9), B2(0.82), B1(0.74).
- Keep B3.
- IoU(B3, B4) = 0.250 ≤ thr → keep for now
- IoU(B3, B2) = 0.161 ≤ thr → keep for now
- IoU(B3, B1) = 0.500 > thr → suppress B1
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- Keep B2.
- Remaining: B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B6. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,1)–(4,7) | 0.66
B2 | (0,1)–(1,8) | 0.82
B3 | (0,1)–(3,4) | 0.95
B4 | (3,2)–(6,5) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

**Answer:** B2,B3,B4
(Also accepted: B2, B3, B4, B3,B2,B4)

**Working:**

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B2(0.82), B4(0.74), B1(0.66).
- Keep B3.
- IoU(B3, B2) = 0.231 ≤ thr → keep for now
- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- IoU(B3, B1) = 0.375 > thr → suppress B1
- Keep B2.
- IoU(B2, B4) = 0.000 ≤ thr → keep for now
- Keep B4.

- Remaining: B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B7. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,2)–(5,3) | 0.9
B2 | (1,2)–(4,4) | 0.82
B3 | (3,2)–(4,4) | 0.95
B4 | (2,2)–(4,4) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

**Answer:** B1,B2,B3
(Also accepted: B1, B2, B3, B3,B1,B2)

**Working:**

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B1(0.9), B2(0.82), B4(0.74).
- Keep B3.
- IoU(B3, B1) = 0.167 ≤ thr → keep for now
- IoU(B3, B2) = 0.333 ≤ thr → keep for now
- IoU(B3, B4) = 0.500 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B2) = 0.375 ≤ thr → keep for now
- IoU(B1, B4) = 0.286 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B4) = 0.667 > thr → suppress B4
- Remaining: B1,B2,B3.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B8. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,2)–(6,8) | 0.74
B2 | (0,3)–(9,9) | 0.95
B3 | (1,1)–(8,9) | 0.58
B4 | (5,5)–(7,8) | 0.66
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

**Answer:** B2,B4
(Also accepted: B2, B4)

**Working:**

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B1(0.74), B4(0.66), B3(0.58).
- Keep B2.
- IoU(B2, B1) = 0.500 > thr → suppress B1
- IoU(B2, B4) = 0.111 ≤ thr → keep for now
- IoU(B2, B3) = 0.618 > thr → suppress B3
- Keep B4.
- Remaining: B2,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

## DETECT B9. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (3,3)–(8,6) | 0.74
B2 | (6,0)–(9,7) | 0.95
B3 | (6,1)–(8,7) | 0.66
B4 | (3,6)–(8,10) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

**Answer:** B1,B2,B4
(Also accepted: B1, B2, B4, B2,B4,B1)

**Working:**

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B4(0.9), B1(0.74), B3(0.66).
- Keep B2.
- IoU(B2, B4) = 0.051 ≤ thr → keep for now
- IoU(B2, B1) = 0.200 ≤ thr → keep for now
- IoU(B2, B3) = 0.571 > thr → suppress B3
- Keep B4.
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B1.
- Remaining: B1,B2,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

## DETECT B10. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (6,6)–(11,9) | 0.82
B2 | (5,6)–(9,8) | 0.95
B3 | (5,4)–(7,7) | 0.9
B4 | (1,6)–(4,8) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

**Answer:** B2,B3,B4
(Also accepted: B2, B3, B4)

**Working:**

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B3(0.9), B1(0.82), B4(0.74).
- Keep B2.
- IoU(B2, B3) = 0.167 ≤ thr → keep for now
- IoU(B2, B1) = 0.353 > thr → suppress B1
- IoU(B2, B4) = 0.000 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- Keep B4.
- Remaining: B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B11. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (1,0)–(10,9) | 0.74
B2 | (4,1)–(9,6) | 0.58
B3 | (4,3)–(8,6) | 0.82
B4 | (4,1)–(6,4) | 0.66
B5 | (1,6)–(6,8) | 0.95
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

**Answer:** B1,B2,B3,B4,B5
(Also accepted: B1, B2, B3, B4, B5, B5,B3,B1,B4,B2)

**Working:**

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B5(0.95), B3(0.82), B1(0.74), B4(0.66), B2(0.58).
- Keep B5.
- IoU(B5, B3) = 0.000 ≤ thr → keep for now
- IoU(B5, B1) = 0.123 ≤ thr → keep for now
- IoU(B5, B4) = 0.000 ≤ thr → keep for now
- IoU(B5, B2) = 0.000 ≤ thr → keep for now
- Keep B3.

- IoU(B3, B1) = 0.148 ≤ thr → keep for now
- IoU(B3, B4) = 0.125 ≤ thr → keep for now
- IoU(B3, B2) = 0.480 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B4) = 0.074 ≤ thr → keep for now
- IoU(B1, B2) = 0.309 ≤ thr → keep for now
- Keep B4.
- IoU(B4, B2) = 0.240 ≤ thr → keep for now
- Keep B2.
- Remaining: B1,B2,B3,B4,B5.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B12. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,0)–(8,1) | 0.82
B2 | (0,0)–(6,3) | 0.9
B3 | (1,0)–(8,5) | 0.74
B4 | (5,4)–(9,6) | 0.95
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B4,B2,B1,B3)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.95), B2(0.9), B1(0.82), B3(0.74).
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- IoU(B4, B3) = 0.075 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.250 ≤ thr → keep for now
- IoU(B2, B3) = 0.395 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B3) = 0.200 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B13. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,0)–(8,7) | 0.82
B2 | (4,0)–(10,8) | 0.9
B3 | (1,0)–(10,5) | 0.95
B4 | (5,1)–(10,6) | 0.66
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B3,B4
(Also accepted: B3, B4)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B2(0.9), B1(0.82), B4(0.66).
- Keep B3.
- IoU(B3, B2) = 0.476 > thr → suppress B2
- IoU(B3, B1) = 0.593 > thr → suppress B1
- IoU(B3, B4) = 0.400 ≤ thr → keep for now
- Keep B4.
- Remaining: B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B14. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (4,6)–(10,8) | 0.82
B2 | (6,5)–(8,9) | 0.9
B3 | (6,6)–(8,7) | 0.58
B4 | (6,3)–(11,8) | 0.66
B5 | (5,6)–(10,8) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B2,B1,B4,B3)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.9), B1(0.82), B5(0.74), B4(0.66), B3(0.58).
- Keep B2.
- IoU(B2, B1) = 0.250 ≤ thr → keep for now
- IoU(B2, B5) = 0.286 ≤ thr → keep for now
- IoU(B2, B4) = 0.222 ≤ thr → keep for now
- IoU(B2, B3) = 0.250 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B5) = 0.833 > thr → suppress B5
- IoU(B1, B4) = 0.276 ≤ thr → keep for now

- IoU(B1, B3) = 0.167 ≤ thr → keep for now
- Keep B4.
- IoU(B4, B3) = 0.080 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B15. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (7,2)–(11,8) | 0.58
B2 | (7,2)–(8,10) | 0.66
B3 | (6,6)–(8,10) | 0.74
B4 | (1,5)–(6,10) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

**Answer:** B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B4,B3,B2,B1)

**Working:**

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B3(0.74), B2(0.66), B1(0.58).
- Keep B4.
- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B2) = 0.333 ≤ thr → keep for now
- IoU(B3, B1) = 0.067 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.231 ≤ thr → keep for now
- Keep B1.
- Remaining: B1,B2,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

### DETECT B16. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,1)–(6,7) | 0.9
B2 | (0,0)–(7,8) | 0.66
B3 | (2,3)–(4,4) | 0.95
B4 | (0,0)–(5,2) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

**Answer:** B1,B3,B4
(Also accepted: B1, B3, B4, B3,B1,B4)

**Working:**

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B1(0.9), B4(0.74), B2(0.66).
- Keep B3.
- IoU(B3, B1) = 0.067 ≤ thr → keep for now
- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- IoU(B3, B2) = 0.036 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B4) = 0.111 ≤ thr → keep for now
- IoU(B1, B2) = 0.536 > thr → suppress B2
- Keep B4.
- Remaining: B1,B3,B4.

**Takeaway:** NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

*Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression*

---

# 🔁 Sequence Models & NLP (Lecture 8)

## Part B: calculation drills (16)

### SEQ B1. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 6 | 7
delightful | 4 | 8
slow | 0 | 0
repetitive | 3 | 3
charming | 8 | 3
dull | 5 | 3

Training data: 5 positive and 6 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 26$, $T_- = 24$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.458; class = negative

**Working:**

- Priors: P(+) = 5/11 = 0.4545, P(−) = 0.5455.
- heartwarming: P(w|+) = (6+1)/(26+6) = 0.2188; P(w|−) = (7+1)/(24+6) = 0.2667
- slow: P(w|+) = (0+1)/(26+6) = 0.0313; P(w|−) = (0+1)/(24+6) = 0.0333
- repetitive: P(w|+) = (3+1)/(26+6) = 0.1250; P(w|−) = (3+1)/(24+6) = 0.1333
- dull: P(w|+) = (5+1)/(26+6) = 0.1875; P(w|−) = (3+1)/(24+6) = 0.1333
- $S_+ = 0.4545 \times 0.2188 \times 0.0313 \times 0.1250 \times 0.1875 = 7.2826e\text{-}5$.
- $S_- = 0.5455 \times 0.2667 \times 0.0333 \times 0.1333 \times 0.1333 = 8.6195e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4580 \rightarrow$ negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

### SEQ B2. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 4
delightful | 2 | 1
slow | 2 | 2
repetitive | 3 | 6
charming | 7 | 5
dull | 4 | 6

Training data: 12 positive and 6 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 20$, $T_- = 24$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.618; class = positive

**Working:**

- Priors: P(+) = 12/18 = 0.6667, P(−) = 0.3333.
- heartwarming: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (4+1)/(24+6) = 0.1667
- slow: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (2+1)/(24+6) = 0.1000
- repetitive: P(w|+) = (3+1)/(20+6) = 0.1538; P(w|−) = (6+1)/(24+6) = 0.2333
- charming: P(w|+) = (7+1)/(20+6) = 0.3077; P(w|−) = (5+1)/(24+6) = 0.2000
- $S_+$ = 0.6667 × 0.1154 × 0.1154 × 0.1538 × 0.3077 = 4.2015e-4.
- $S_-$ = 0.3333 × 0.1667 × 0.1000 × 0.2333 × 0.2000 = 2.5926e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.6184 → positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B3. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 8 | 1
delightful | 2 | 3
slow | 6 | 1
repetitive | 4 | 4
charming | 4 | 7
dull | 6 | 3

Training data: 10 positive and 13 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 30, $T_-$ = 19. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, delightful, slow, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.787; class = positive

**Working:**

- Priors: P(+) = 10/23 = 0.4348, P(−) = 0.5652.
- heartwarming: P(w|+) = (8+1)/(30+6) = 0.2500; P(w|−) = (1+1)/(19+6) = 0.0800
- delightful: P(w|+) = (2+1)/(30+6) = 0.0833; P(w|−) = (3+1)/(19+6) = 0.1600
- slow: P(w|+) = (6+1)/(30+6) = 0.1944; P(w|−) = (1+1)/(19+6) = 0.0800
- dull: P(w|+) = (6+1)/(30+6) = 0.1944; P(w|−) = (3+1)/(19+6) = 0.1600
- $S_+$ = 0.4348 × 0.2500 × 0.0833 × 0.1944 × 0.1944 = 3.4247e-4.
- $S_-$ = 0.5652 × 0.0800 × 0.1600 × 0.0800 × 0.1600 = 9.2605e-5.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.7872 → positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B4. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 3 | 4
delightful | 3 | 2
slow | 4 | 7

repetitive | 7 | 2
charming | 8 | 4
dull | 1 | 6

Training data: 10 positive and 12 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 26$, $T_- = 25$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.694; class = positive

**Working:**

- Priors: P(+) = 10/22 = 0.4545, P(−) = 0.5455.
- slow: P(w|+) = (4+1)/(26+6) = 0.1563; P(w|−) = (7+1)/(25+6) = 0.2581
- repetitive: P(w|+) = (7+1)/(26+6) = 0.2500; P(w|−) = (2+1)/(25+6) = 0.0968
- charming: P(w|+) = (8+1)/(26+6) = 0.2813; P(w|−) = (4+1)/(25+6) = 0.1613
- $S_+ = 0.4545 \times 0.1563 \times 0.2500 \times 0.2813 = 4.9938e\text{-}3$.
- $S_- = 0.5455 \times 0.2581 \times 0.0968 \times 0.1613 = 2.1971e\text{-}3$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.6945 \rightarrow$ positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B5. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 0 | 0
delightful | 4 | 1
slow | 6 | 6
repetitive | 7 | 0
charming | 8 | 8
dull | 2 | 4

Training data: 6 positive and 10 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 27$, $T_- = 19$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, repetitive, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.487; class = negative

**Working:**

- Priors: P(+) = 6/16 = 0.3750, P(−) = 0.6250.
- heartwarming: P(w|+) = (0+1)/(27+6) = 0.0303; P(w|−) = (0+1)/(19+6) = 0.0400
- repetitive: P(w|+) = (7+1)/(27+6) = 0.2424; P(w|−) = (0+1)/(19+6) = 0.0400
- charming: P(w|+) = (8+1)/(27+6) = 0.2727; P(w|−) = (8+1)/(19+6) = 0.3600
- dull: P(w|+) = (2+1)/(27+6) = 0.0909; P(w|−) = (4+1)/(19+6) = 0.2000
- $S_+ = 0.3750 \times 0.0303 \times 0.2424 \times 0.2727 \times 0.0909 = 6.8301e\text{-}5$.
- $S_- = 0.6250 \times 0.0400 \times 0.0400 \times 0.3600 \times 0.2000 = 7.2000e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4868 \rightarrow$ negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B6. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 5 | 4
delightful | 2 | 5
slow | 5 | 4
repetitive | 2 | 1
charming | 3 | 7
dull | 3 | 8

Training data: 11 positive and 15 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 20$, $T_- = 29$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: delightful, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.401; class = negative

**Working:**

- Priors: P(+) = 11/26 = 0.4231, P(−) = 0.5769.
- delightful: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (5+1)/(29+6) = 0.1714
- repetitive: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (1+1)/(29+6) = 0.0571
- charming: P(w|+) = (3+1)/(20+6) = 0.1538; P(w|−) = (7+1)/(29+6) = 0.2286
- $S_+ = 0.4231 \times 0.1154 \times 0.1154 \times 0.1538 = 8.6657e\text{-}4$.
- $S_- = 0.5769 \times 0.1714 \times 0.0571 \times 0.2286 = 1.2918e\text{-}3$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4015 \rightarrow$ negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B7. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 4 | 2
delightful | 4 | 6
slow | 5 | 0
repetitive | 8 | 8
charming | 6 | 6
dull | 8 | 8

Training data: 15 positive and 12 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 35$, $T_- = 30$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.491; class = negative

**Working:**

- Priors: P(+) = 15/27 = 0.5556, P(−) = 0.4444.
- repetitive: P(w|+) = (8+1)/(35+6) = 0.2195; P(w|−) = (8+1)/(30+6) = 0.2500
- dull: P(w|+) = (8+1)/(35+6) = 0.2195; P(w|−) = (8+1)/(30+6) = 0.2500

- $S_+$ = 0.5556 × 0.2195 × 0.2195 = 2.6770e-2.
- $S_-$ = 0.4444 × 0.2500 × 0.2500 = 2.7778e-2.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.4908 → negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B8. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 8 | 7
delightful | 0 | 7
slow | 0 | 8
repetitive | 7 | 4
charming | 3 | 8
dull | 4 | 5

Training data: 5 positive and 5 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 22, $T_-$ = 39. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.236; class = negative

**Working:**

- Priors: P(+) = 5/10 = 0.5000, P(−) = 0.5000.
- heartwarming: P(w|+) = (8+1)/(22+6) = 0.3214; P(w|−) = (7+1)/(39+6) = 0.1778
- slow: P(w|+) = (0+1)/(22+6) = 0.0357; P(w|−) = (8+1)/(39+6) = 0.2000
- charming: P(w|+) = (3+1)/(22+6) = 0.1429; P(w|−) = (8+1)/(39+6) = 0.2000
- dull: P(w|+) = (4+1)/(22+6) = 0.1786; P(w|−) = (5+1)/(39+6) = 0.1333
- $S_+$ = 0.5000 × 0.3214 × 0.0357 × 0.1429 × 0.1786 = 1.4642e-4.
- $S_-$ = 0.5000 × 0.1778 × 0.2000 × 0.2000 × 0.1333 = 4.7407e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.2360 → negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B9. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 6
delightful | 0 | 2
slow | 6 | 8
repetitive | 3 | 3
charming | 0 | 7
dull | 3 | 1

Training data: 14 positive and 9 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 14, $T_-$ = 27. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: delightful, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.226; class = negative

**Working:**

- Priors: P(+) = 14/23 = 0.6087, P(−) = 0.3913.
- delightful: P(w|+) = (0+1)/(14+6) = 0.0500; P(w|−) = (2+1)/(27+6) = 0.0909
- repetitive: P(w|+) = (3+1)/(14+6) = 0.2000; P(w|−) = (3+1)/(27+6) = 0.1212
- charming: P(w|+) = (0+1)/(14+6) = 0.0500; P(w|−) = (7+1)/(27+6) = 0.2424
- $S_+$ = 0.6087 × 0.0500 × 0.2000 × 0.0500 = 3.0435e-4.
- $S_-$ = 0.3913 × 0.0909 × 0.1212 × 0.2424 = 1.0453e-3.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.2255 → negative.

**Takeaway:** Naive Bayes: P(c|text) ∝ P(c)·$\Pi_i$ P($x_i$|c). Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B10. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 5
delightful | 0 | 2
slow | 1 | 8
repetitive | 5 | 0
charming | 4 | 8
dull | 0 | 3

Training data: 9 positive and 5 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 17, $T_-$ = 26. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.869; class = positive

**Working:**

- Priors: P(+) = 9/14 = 0.6429, P(−) = 0.3571.
- heartwarming: P(w|+) = (7+1)/(17+6) = 0.3478; P(w|−) = (5+1)/(26+6) = 0.1875
- slow: P(w|+) = (1+1)/(17+6) = 0.0870; P(w|−) = (8+1)/(26+6) = 0.2813
- repetitive: P(w|+) = (5+1)/(17+6) = 0.2609; P(w|−) = (0+1)/(26+6) = 0.0313
- charming: P(w|+) = (4+1)/(17+6) = 0.2174; P(w|−) = (8+1)/(26+6) = 0.2813
- $S_+$ = 0.6429 × 0.3478 × 0.0870 × 0.2609 × 0.2174 = 1.1027e-3.
- $S_-$ = 0.3571 × 0.1875 × 0.2813 × 0.0313 × 0.2813 = 1.6553e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.8695 → positive.

**Takeaway:** Naive Bayes: P(c|text) ∝ P(c)·$\Pi_i$ P($x_i$|c). Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B11. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 6
delightful | 6 | 0
slow | 3 | 4
repetitive | 1 | 0

charming | 3 | 5
dull | 0 | 1

Training data: 13 positive and 15 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 15$, $T_- = 16$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.214; class = negative

**Working:**

- Priors: P(+) = 13/28 = 0.4643, P(−) = 0.5357.
- heartwarming: P(w|+) = (2+1)/(15+6) = 0.1429; P(w|−) = (6+1)/(16+6) = 0.3182
- charming: P(w|+) = (3+1)/(15+6) = 0.1905; P(w|−) = (5+1)/(16+6) = 0.2727
- $S_+ = 0.4643 \times 0.1429 \times 0.1905 = 1.2634e\text{-}2$.
- $S_- = 0.5357 \times 0.3182 \times 0.2727 = 4.6488e\text{-}2$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.2137 \rightarrow$ negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B12. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 8
delightful | 3 | 7
slow | 1 | 0
repetitive | 4 | 0
charming | 0 | 1
dull | 6 | 6

Training data: 15 positive and 14 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 21$, $T_- = 22$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.857; class = positive

**Working:**

- Priors: P(+) = 15/29 = 0.5172, P(−) = 0.4828.
- slow: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (0+1)/(22+6) = 0.0357
- repetitive: P(w|+) = (4+1)/(21+6) = 0.1852; P(w|−) = (0+1)/(22+6) = 0.0357
- charming: P(w|+) = (0+1)/(21+6) = 0.0370; P(w|−) = (1+1)/(22+6) = 0.0714
- $S_+ = 0.5172 \times 0.0741 \times 0.1852 \times 0.0370 = 2.6279e\text{-}4$.
- $S_- = 0.4828 \times 0.0357 \times 0.0357 \times 0.0714 = 4.3983e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.8566 \rightarrow$ positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B13. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 4 | 0
delightful | 3 | 4
slow | 3 | 8
repetitive | 3 | 6
charming | 8 | 5
dull | 5 | 3

Training data: 15 positive and 9 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 26$, $T_- = 26$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.893; class = positive

**Working:**

- Priors: P(+) = 15/24 = 0.6250, P(−) = 0.3750.
- heartwarming: P(w|+) = (4+1)/(26+6) = 0.1563; P(w|−) = (0+1)/(26+6) = 0.0313
- slow: P(w|+) = (3+1)/(26+6) = 0.1250; P(w|−) = (8+1)/(26+6) = 0.2813
- charming: P(w|+) = (8+1)/(26+6) = 0.2813; P(w|−) = (5+1)/(26+6) = 0.1875
- dull: P(w|+) = (5+1)/(26+6) = 0.1875; P(w|−) = (3+1)/(26+6) = 0.1250
- $S_+ = 0.6250 \times 0.1563 \times 0.1250 \times 0.2813 \times 0.1875 = 6.4373e\text{-}4$.
- $S_- = 0.3750 \times 0.0313 \times 0.2813 \times 0.1875 \times 0.1250 = 7.7248e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.8929 \rightarrow$ positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B14. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 2
delightful | 7 | 1
slow | 6 | 4
repetitive | 5 | 2
charming | 2 | 2
dull | 6 | 4

Training data: 8 positive and 6 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 28$, $T_- = 15$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, delightful, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.521; class = positive

**Working:**

- Priors: P(+) = 8/14 = 0.5714, P(−) = 0.4286.
- heartwarming: P(w|+) = (2+1)/(28+6) = 0.0882; P(w|−) = (2+1)/(15+6) = 0.1429

- delightful: P(w|+) = (7+1)/(28+6) = 0.2353; P(w|−) = (1+1)/(15+6) = 0.0952
- charming: P(w|+) = (2+1)/(28+6) = 0.0882; P(w|−) = (2+1)/(15+6) = 0.1429
- dull: P(w|+) = (6+1)/(28+6) = 0.2059; P(w|−) = (4+1)/(15+6) = 0.2381
- $S_+$ = 0.5714 × 0.0882 × 0.2353 × 0.0882 × 0.2059 = 2.1551e-4.
- $S_-$ = 0.4286 × 0.1429 × 0.0952 × 0.1429 × 0.2381 = 1.9833e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.5208 → positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B15. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 6
delightful | 3 | 0
slow | 3 | 6
repetitive | 1 | 6
charming | 2 | 1
dull | 2 | 0

Training data: 6 positive and 7 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 18, $T_-$ = 19. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, delightful, slow, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.798; class = positive

**Working:**

- Priors: P(+) = 6/13 = 0.4615, P(−) = 0.5385.
- heartwarming: P(w|+) = (7+1)/(18+6) = 0.3333; P(w|−) = (6+1)/(19+6) = 0.2800
- delightful: P(w|+) = (3+1)/(18+6) = 0.1667; P(w|−) = (0+1)/(19+6) = 0.0400
- slow: P(w|+) = (3+1)/(18+6) = 0.1667; P(w|−) = (6+1)/(19+6) = 0.2800
- charming: P(w|+) = (2+1)/(18+6) = 0.1250; P(w|−) = (1+1)/(19+6) = 0.0800
- $S_+$ = 0.4615 × 0.3333 × 0.1667 × 0.1667 × 0.1250 = 5.3419e-4.
- $S_-$ = 0.5385 × 0.2800 × 0.0400 × 0.2800 × 0.0800 = 1.3509e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.7982 → positive.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

## SEQ B16. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 3 | 5
delightful | 1 | 7
slow | 4 | 8
repetitive | 7 | 2
charming | 5 | 0
dull | 1 | 8

Training data: 15 positive and 10 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+$ = 21, $T_-$ = 30. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review

containing: delightful, repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

**Answer:** P(+|text) = 0.345; class = negative

**Working:**

- Priors: P(+) = 15/25 = 0.6000, P(−) = 0.4000.
- delightful: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (7+1)/(30+6) = 0.2222
- repetitive: P(w|+) = (7+1)/(21+6) = 0.2963; P(w|−) = (2+1)/(30+6) = 0.0833
- dull: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (8+1)/(30+6) = 0.2500
- $S_+$ = 0.6000 × 0.0741 × 0.2963 × 0.0741 = 9.7546e-4.
- $S_-$ = 0.4000 × 0.2222 × 0.0833 × 0.2500 = 1.8519e-3.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.3450 → negative.

**Takeaway:** Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

*Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update*

---

# 🎯 Attention & Transformers (Lecture 8)

## Part B: calculation drills (16)

### ATTN B1. [BLANKS · Exam+ · calculation]

Query q = [1, 3]; keys k1 = [2, 0], k2 = [2, 3]; values v1 = [1, 0], v2 = [4, 1]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.002; out[1] = 3.995; out[2] = 0.998

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 0] | [1, 0] | 2 | 1.414 | 4.113 | 0.002
  k2 | [2, 3] | [4, 1] | 11 | 7.778 | 2387.912 | 0.998
- Output = $\Sigma w_i v_i$ = [3.9948, 0.9983].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B2. [BLANKS · Exam+ · calculation]

Query q = [0, 0]; keys k1 = [-2, 2], k2 = [3, -2]; values v1 = [4, 1], v2 = [3, 2]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.500; out[1] = 3.500; out[2] = 1.500

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-2, 2] | [4, 1] | 0 | 0.000 | 1.000 | 0.500
  k2 | [3, -2] | [3, 2] | 0 | 0.000 | 1.000 | 0.500
- Output = $\Sigma w_i v_i$ = [3.5000, 1.5000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B3. [BLANKS · Exam+ · calculation]

Query q = [0, -2]; keys k1 = [2, 0], k2 = [1, 3], k3 = [1, 0]; values v1 = [0, 3], v2 = [2, 4], v3 = [4, 4]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.496; out[1] = 2.000; out[2] = 3.504

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 0] | [0, 3] | 0 | 0.000 | 1.000 | 0.496
  k2 | [1, 3] | [2, 4] | -6 | -4.243 | 0.014 | 0.007
  k3 | [1, 0] | [4, 4] | 0 | 0.000 | 1.000 | 0.496
- Output = $\Sigma w_i v_i$ = [2.0000, 3.5036].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B4. [BLANKS · Exam+ · calculation]

Query q = [1, -1]; keys k1 = [-1, 1], k2 = [-2, -1]; values v1 = [0, 2], v2 = [1, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}) \cdot V$. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.330; out[1] = 0.670; out[2] = 3.340

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-1, 1] | [0, 2] | -2 | -1.414 | 0.243 | 0.330
  k2 | [-2, -1] | [1, 4] | -1 | -0.707 | 0.493 | 0.670
- Output = $\Sigma w_i v_i$ = [0.6698, 3.3395].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B5. [BLANKS · Exam+ · calculation]

Query q = [0, 1]; keys k1 = [-2, 1], k2 = [1, -2]; values v1 = [-1, 1], v2 = [1, 1]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}) \cdot V$. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.893; out[1] = -0.786; out[2] = 1.000

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-2, 1] | [-1, 1] | 1 | 0.707 | 2.028 | 0.893
  k2 | [1, -2] | [1, 1] | -2 | -1.414 | 0.243 | 0.107
- Output = $\Sigma w_i v_i$ = [-0.7859, 1.0000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B6. [BLANKS · Exam+ · calculation]

Query q = [-1, 0]; keys k1 = [1, 1], k2 = [-1, 1], k3 = [3, 2]; values v1 = [1, 1], v2 = [0, -1], v3 = [2, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}) \cdot V$. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.187; out[1] = 0.277; out[2] = -0.400

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [1, 1] | [1, 1] | -1 | -0.707 | 0.493 | 0.187
  k2 | [-1, 1] | [0, -1] | 1 | 0.707 | 2.028 | 0.768
  k3 | [3, 2] | [2, 4] | -3 | -2.121 | 0.120 | 0.045
- Output = $\Sigma\, w_i v_i$ = [0.2775, -0.3997].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B7. [BLANKS · Exam+ · calculation]

Query q = [0, 2]; keys k1 = [0, -1], k2 = [-2, 0]; values v1 = [3, -1], v2 = [3, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $\text{weight}_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.196; out[1] = 3.000; out[2] = 3.022

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [0, -1] | [3, -1] | -2 | -1.414 | 0.243 | 0.196
  k2 | [-2, 0] | [3, 4] | 0 | 0.000 | 1.000 | 0.804
- Output = $\Sigma\, w_i v_i$ = [3.0000, 3.0221].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B8. [BLANKS · Exam+ · calculation]

Query q = [1, -2]; keys k1 = [2, 3], k2 = [1, -1]; values v1 = [3, 4], v2 = [4, 2]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $\text{weight}_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.007; out[1] = 3.993; out[2] = 2.014

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 3] | [3, 4] | -4 | -2.828 | 0.059 | 0.007
  k2 | [1, -1] | [4, 2] | 3 | 2.121 | 8.342 | 0.993
- Output = $\Sigma\, w_i v_i$ = [3.9930, 2.0141].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B9. [BLANKS · Exam+ · calculation]

Query q = [-1, 3]; keys k1 = [-2, -2], k2 = [1, -1]; values v1 = [1, 4], v2 = [3, 3]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $\text{weight}_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.500; out[1] = 2.000; out[2] = 3.500

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-2, -2] | [1, 4] | -4 | -2.828 | 0.059 | 0.500
  k2 | [1, -1] | [3, 3] | -4 | -2.828 | 0.059 | 0.500
- Output = $\Sigma\, w_i v_i$ = [2.0000, 3.5000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

## ATTN B10. [BLANKS · Exam+ · calculation]

Query q = [2, 1]; keys k1 = [-1, 1], k2 = [1, 3], k3 = [0, 3]; values v1 = [-1, 1], v2 = [4, 4], v3 = [3, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $\text{weight}_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.011; out[1] = 3.750; out[2] = 3.966

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-1, 1] | [-1, 1] | -1 | -0.707 | 0.493 | 0.011
  k2 | [1, 3] | [4, 4] | 5 | 3.536 | 34.313 | 0.795
  k3 | [0, 3] | [3, 4] | 3 | 2.121 | 8.342 | 0.193
- Output = $\Sigma\, w_i v_i$ = [3.7495, 3.9657].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

## ATTN B11. [BLANKS · Exam+ · calculation]

Query q = [-2, 2]; keys k1 = [2, 2], k2 = [-1, -1]; values v1 = [-1, 3], v2 = [0, 3]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $\text{weight}_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $\text{weight}_1$ = 0.500; out[1] = -0.500; out[2] = 3.000

**Working:**

- Key | $k_i$ | $v_i$ | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 2] | [-1, 3] | 0 | 0.000 | 1.000 | 0.500
  k2 | [-1, -1] | [0, 3] | 0 | 0.000 | 1.000 | 0.500
- Output = $\Sigma\, w_i v_i$ = [-0.5000, 3.0000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

## ATTN B12. [BLANKS · Exam+ · calculation]

Query q = [-1, 0]; keys k1 = [-2, 0], k2 = [3, 2], k3 = [-2, 2]; values v1 = [2, 3], v2 = [-1, 4], v3 = [1, 2]; d_k = 2. Compute softmax(q·$K^T$/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.493; out[1] = 1.464; out[2] = 2.522

**Working:**

- Key | $k_i$ | $v_i$ | score q·$k_i$ | scaled /√2 | exp | weight
  k1 | [-2, 0] | [2, 3] | 2 | 1.414 | 4.113 | 0.493
  k2 | [3, 2] | [-1, 4] | -3 | -2.121 | 0.120 | 0.014
  k3 | [-2, 2] | [1, 2] | 2 | 1.414 | 4.113 | 0.493
- Output = Σ $w_iv_i$ = [1.4641, 2.5215].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

## ATTN B13. [BLANKS · Exam+ · calculation]

Query q = [2, -1]; keys k1 = [-2, 0], k2 = [3, 1]; values v1 = [1, 0], v2 = [2, 1]; d_k = 2. Compute softmax(q·$K^T$/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.002; out[1] = 1.998; out[2] = 0.998

**Working:**

- Key | $k_i$ | $v_i$ | score q·$k_i$ | scaled /√2 | exp | weight
  k1 | [-2, 0] | [1, 0] | -4 | -2.828 | 0.059 | 0.002
  k2 | [3, 1] | [2, 1] | 5 | 3.536 | 34.313 | 0.998
- Output = Σ $w_iv_i$ = [1.9983, 0.9983].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

## ATTN B14. [BLANKS · Exam+ · calculation]

Query q = [0, -2]; keys k1 = [0, -2], k2 = [2, -1]; values v1 = [4, -1], v2 = [1, -1]; d_k = 2. Compute softmax(q·$K^T$/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.804; out[1] = 3.413; out[2] = -1.000

**Working:**

- Key | $k_i$ | $v_i$ | score q·$k_i$ | scaled /√2 | exp | weight
  k1 | [0, -2] | [4, -1] | 4 | 2.828 | 16.919 | 0.804
  k2 | [2, -1] | [1, -1] | 2 | 1.414 | 4.113 | 0.196
- Output = Σ $w_iv_i$ = [3.4133, -1.0000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B15. [BLANKS · Exam+ · calculation]

Query q = [-1, -2]; keys k1 = [1, 2], k2 = [-2, 2], k3 = [-2, 1]; values v1 = [3, -1], v2 = [3, -1], v3 = [4, 2]; d_k = 2. Compute softmax(q·$K^T$/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.023; out[1] = 3.786; out[2] = 1.358

**Working:**

- Key | $k_i$ | $v_i$ | score q·$k_i$ | scaled /√2 | exp | weight
  k1 | [1, 2] | [3, -1] | -5 | -3.536 | 0.029 | 0.023
  k2 | [-2, 2] | [3, -1] | -2 | -1.414 | 0.243 | 0.191
  k3 | [-2, 1] | [4, 2] | 0 | 0.000 | 1.000 | 0.786
- Output = Σ $w_iv_i$ = [3.7860, 1.3580].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

### ATTN B16. [BLANKS · Exam+ · calculation]

Query q = [2, -1]; keys k1 = [1, 3], k2 = [1, -2], k3 = [2, 3]; values v1 = [2, 2], v2 = [-1, 2], v3 = [2, 2]; d_k = 2. Compute softmax(q·$K^T$/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

**Answer:** $weight_1$ = 0.025; out[1] = -0.611; out[2] = 2.000

**Working:**

- Key | $k_i$ | $v_i$ | score q·$k_i$ | scaled /√2 | exp | weight
  k1 | [1, 3] | [2, 2] | -1 | -0.707 | 0.493 | 0.025
  k2 | [1, -2] | [-1, 2] | 4 | 2.828 | 16.919 | 0.870
  k3 | [2, 3] | [2, 2] | 1 | 0.707 | 2.028 | 0.104
- Output = Σ $w_iv_i$ = [-0.6109, 2.0000].

**Takeaway:** Scores → scale by √d_k → row softmax → weighted sum of values.

*Source: Lecture 8 (Self-attention) + Key points about Attention*

---

# 🧠 Neural Networks (Workshop 5)

## Part B: calculation drills (16)

### NN B1. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.21$, $a_4 = 0.24$; $w_{3,5} = 0.5$, $w_{4,5} = 1$, $w_{0,5} = 1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.7933$; $\Delta_5 = -0.0678$; $\partial L/\partial w_{3,5} = -0.0142$

**Working:**

- $in_5 = 0.5\times0.21 + 1\times0.24 + 1 = 1.3450$; $\hat{y} = \sigma(in_5) = 0.7933$.
- $\Delta_5 = 2(0.7933 - 1)\times0.7933\times(1 - 0.7933) = -0.0678$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = -0.0678\times0.21 = -0.0142$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.0056$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B2. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.4$, $a_4 = 0.5$; $w_{3,5} = -1.5$, $w_{4,5} = 2$, $w_{0,5} = 0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.7109$; $\Delta_5 = 0.2922$; $\partial L/\partial w_{3,5} = 0.1169$

**Working:**

- $in_5 = -1.5\times0.4 + 2\times0.5 + 0.5 = 0.9000$; $\hat{y} = \sigma(in_5) = 0.7109$.
- $\Delta_5 = 2(0.7109 - 0)\times0.7109\times(1 - 0.7109) = 0.2922$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2922\times0.4 = 0.1169$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.1052$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B3. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.64$, $a_4 = 0.61$; $w_{3,5} = -1$, $w_{4,5} = 0.5$, $w_{0,5} = -1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.2083$; $\Delta_5 = 0.0687$; $\partial L/\partial w_{3,5} = 0.0440$

**Working:**

- $in_5 = -1\times0.64 + 0.5\times0.61 + -1 = -1.3350$; $\hat{y} = \sigma(in_5) = 0.2083$.
- $\Delta_5 = 2(0.2083 - 0)\times0.2083\times(1 - 0.2083) = 0.0687$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.0687\times0.64 = 0.0440$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.0158$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B4. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.73$, $a_4 = 0.23$; $w_{3,5} = 1.5$, $w_{4,5} = 1.5$, $w_{0,5} = 0$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = ____
(2) $\Delta_5$ = ____
(3) $\partial L/\partial w_{3,5}$ = ____

**Answer:** $\hat{y} = 0.8085$; $\Delta_5 = 0.2504$; $\partial L/\partial w_{3,5} = 0.1828$

**Working:**

- $in_5 = 1.5\times0.73 + 1.5\times0.23 + 0 = 1.4400$; $\hat{y} = \sigma(in_5) = 0.8085$.
- $\Delta_5 = 2(0.8085 - 0)\times0.8085\times(1 - 0.8085) = 0.2504$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2504\times0.73 = 0.1828$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0740$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B5. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.59$, $a_4 = 0.43$; $w_{3,5} = 2$, $w_{4,5} = 1$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = ____
(2) $\Delta_5$ = ____
(3) $\partial L/\partial w_{3,5}$ = ____

**Answer:** $\hat{y} = 0.9315$; $\Delta_5 = 0.1189$; $\partial L/\partial w_{3,5} = 0.0701$

**Working:**

- $in_5 = 2\times0.59 + 1\times0.43 + 1 = 2.6100$; $\hat{y} = \sigma(in_5) = 0.9315$.
- $\Delta_5 = 2(0.9315 - 0)\times0.9315\times(1 - 0.9315) = 0.1189$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.1189\times0.59 = 0.0701$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0575$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B6. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.51$, $a_4 = 0.25$; $w_{3,5} = 1.5$, $w_{4,5} = -0.5$, $w_{0,5} = -1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and

$\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y}$ = 0.4110; $\Delta_5$ = -0.2852; $\partial L/\partial w_{3,5}$ = -0.1454

**Working:**

- $in_5$ = 1.5×0.51 + -0.5×0.25 + -1 = -0.3600; $\hat{y} = \sigma(in_5)$ = 0.4110.
- $\Delta_5$ = 2(0.4110 − 1)×0.4110×(1 − 0.4110) = -0.2852.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = -0.2852×0.51 = -0.1454.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = -0.1069, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B7. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3$ = 0.51, $a_4$ = 0.27; $w_{3,5}$ = -0.5, $w_{4,5}$ = -2, $w_{0,5}$ = -0.5; target y = 0. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y}$ = 0.2150; $\Delta_5$ = 0.0726; $\partial L/\partial w_{3,5}$ = 0.0370

**Working:**

- $in_5$ = -0.5×0.51 + -2×0.27 + -0.5 = -1.2950; $\hat{y} = \sigma(in_5)$ = 0.2150.
- $\Delta_5$ = 2(0.2150 − 0)×0.2150×(1 − 0.2150) = 0.0726.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.0726×0.51 = 0.0370.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = -0.0091, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B8. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3$ = 0.38, $a_4$ = 0.45; $w_{3,5}$ = 0.5, $w_{4,5}$ = 1, $w_{0,5}$ = 0; target y = 0. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y}$ = 0.6548; $\Delta_5$ = 0.2960; $\partial L/\partial w_{3,5}$ = 0.1125

**Working:**

- $in_5$ = 0.5×0.38 + 1×0.45 + 0 = 0.6400; $\hat{y} = \sigma(in_5)$ = 0.6548.
- $\Delta_5$ = 2(0.6548 − 0)×0.6548×(1 − 0.6548) = 0.2960.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.2960×0.38 = 0.1125.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = 0.0349, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B9. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.6$, $a_4 = 0.26$; $w_{3,5} = 2$, $w_{4,5} = -0.5$, $w_{0,5} = 0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.8278$; $\Delta_5 = 0.2360$; $\partial L/\partial w_{3,5} = 0.1416$

**Working:**

- $in_5 = 2\times0.6 + -0.5\times0.26 + 0.5 = 1.5700$; $\hat{y} = \sigma(in_5) = 0.8278$.
- $\Delta_5 = 2(0.8278 - 0)\times0.8278\times(1 - 0.8278) = 0.2360$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2360\times0.6 = 0.1416$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.1133$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B10. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.53$, $a_4 = 0.53$; $w_{3,5} = -1$, $w_{4,5} = -1$, $w_{0,5} = -0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.1736$; $\Delta_5 = 0.0498$; $\partial L/\partial w_{3,5} = 0.0264$

**Working:**

- $in_5 = -1\times0.53 + -1\times0.53 + -0.5 = -1.5600$; $\hat{y} = \sigma(in_5) = 0.1736$.
- $\Delta_5 = 2(0.1736 - 0)\times0.1736\times(1 - 0.1736) = 0.0498$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.0498\times0.53 = 0.0264$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.0124$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B11. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.53$, $a_4 = 0.35$; $w_{3,5} = 0.5$, $w_{4,5} = 2$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.8771$; $\Delta_5 = 0.1891$; $\partial L/\partial w_{3,5} = 0.1002$

**Working:**

- $in_5 = 0.5\times0.53 + 2\times0.35 + 1 = 1.9650$; $\hat{y} = \sigma(in_5) = 0.8771$.
- $\Delta_5 = 2(0.8771 - 0)\times0.8771\times(1 - 0.8771) = 0.1891$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.1891\times0.53 = 0.1002$.

- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0236$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B12. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.63$, $a_4 = 0.38$; $w_{3,5} = 1$, $w_{4,5} = -1.5$, $w_{0,5} = 0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.6365$; $\Delta_5 = 0.2945$; $\partial L/\partial w_{3,5} = 0.1856$

**Working:**

- $in_5 = 1 \times 0.63 + -1.5 \times 0.38 + 0.5 = 0.5600$; $\hat{y} = \sigma(in_5) = 0.6365$.
- $\Delta_5 = 2(0.6365 - 0) \times 0.6365 \times (1 - 0.6365) = 0.2945$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2945 \times 0.63 = 0.1856$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0687$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B13. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.63$, $a_4 = 0.36$; $w_{3,5} = 0.5$, $w_{4,5} = 0.5$, $w_{0,5} = -1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.3764$; $\Delta_5 = 0.1767$; $\partial L/\partial w_{3,5} = 0.1113$

**Working:**

- $in_5 = 0.5 \times 0.63 + 0.5 \times 0.36 + -1 = -0.5050$; $\hat{y} = \sigma(in_5) = 0.3764$.
- $\Delta_5 = 2(0.3764 - 0) \times 0.3764 \times (1 - 0.3764) = 0.1767$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.1767 \times 0.63 = 0.1113$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0206$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

### NN B14. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.69$, $a_4 = 0.4$; $w_{3,5} = -0.5$, $w_{4,5} = 0.5$, $w_{0,5} = 1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.7016$; $\Delta_5 = -0.1249$; $\partial L/\partial w_{3,5} = -0.0862$

**Working:**

- $in_5 = -0.5\times0.69 + 0.5\times0.4 + 1 = 0.8550$; $\hat{y} = \sigma(in_5) = 0.7016$.
- $\Delta_5 = 2(0.7016 - 1)\times0.7016\times(1 - 0.7016) = -0.1249$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = -0.1249\times0.69 = -0.0862$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0134$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B15. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.62$, $a_4 = 0.71$; $w_{3,5} = -2$, $w_{4,5} = 1.5$, $w_{0,5} = -1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.2360$; $\Delta_5 = -0.2755$; $\partial L/\partial w_{3,5} = -0.1708$

**Working:**

- $in_5 = -2\times0.62 + 1.5\times0.71 + -1 = -1.1750$; $\hat{y} = \sigma(in_5) = 0.2360$.
- $\Delta_5 = 2(0.2360 - 1)\times0.2360\times(1 - 0.2360) = -0.2755$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = -0.2755\times0.62 = -0.1708$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.1298$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*

## NN B16. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.78$, $a_4 = 0.71$; $w_{3,5} = 1$, $w_{4,5} = -0.5$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, $\Delta_5$ and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) $\Delta_5$ = _____
(3) $\partial L/\partial w_{3,5}$ = _____

**Answer:** $\hat{y} = 0.8061$; $\Delta_5 = 0.2520$; $\partial L/\partial w_{3,5} = 0.1965$

**Working:**

- $in_5 = 1\times0.78 + -0.5\times0.71 + 1 = 1.4250$; $\hat{y} = \sigma(in_5) = 0.8061$.
- $\Delta_5 = 2(0.8061 - 0)\times0.8061\times(1 - 0.8061) = 0.2520$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2520\times0.78 = 0.1965$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0432$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

**Takeaway:** Gradient for weight $w_{i,j}$ = (activation coming in, $a_i$) × (perceived error at the receiving unit, $\Delta_j$).

*Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)*